
Agentic Harness for Real-World Compilers

Yingwei Zheng^{*1} Cong Li^{*2} Shaohua Li³ Yuqun Zhang¹ Zhendong Su²

Abstract

Compilers are critical to modern computing, yet fixing compiler bugs is difficult. While recent large language model (LLM) advancements enable automated bug repair, compiler bugs pose unique challenges due to their complexity, deep cross-domain expertise requirements, and sparse, non-descriptive bug reports, necessitating compiler-specific harnesses. To bridge the gap, we introduce *llvm-harness*, the first harness designed to assist LLM agents in understanding and fixing compiler bugs. Our current focus is on the middle end of LLVM, one of the most widely used compiler infrastructures. Central to *llvm-harness* are agent-friendly LLVM tools, a benchmark *llvm-bench* of 334 reproducible LLVM middle-end bugs, and a tailored mini agent *llvm-autofix-mini* for fixing LLVM middle-end bugs automatically. We evaluate five frontier models and find that they exhibit a performance decline when tackling compiler bugs with the state-of-the-art agent. With *llvm-harness*' enhancement, their performance improves by 62%. Our specialized mini agent *llvm-autofix-mini* further outperforms the *llvm-harness*-enhanced state-of-the-art by 22%. This emphasizes the necessity for specialized harnesses like ours to assist LLMs in compiler engineering tasks. Despite promising results, our expert review also reveals several open challenges that remain when applying LLMs for compiler engineering tasks. GitHub: <https://github.com/dtcxzyw/llvm-harness>

1. Introduction

Compilers serve as the foundational infrastructure for modern computing. They translate source code into efficient

machine instructions. Virtually every piece of software, ranging from low-level OS kernels, machine learning (ML) frameworks, to high-level web applications and ML models, relies on them for correct compilation and performance optimization. Hence, it is critical to repair and maintain compilers as soon as any issues arise. Recent advances in large language models (LLMs) present great potential in this direction (Yang et al., 2024b; Mathai et al., 2025).

However, unlike common software, compilers represent a unique category of large-scale, complex software systems. Their internal complexity, marked by sophisticated intermediate representation (IR) and numerous optimizations, renders them exceptionally challenging to maintain and repair. Common software bugs, such as Figure 1a, typically include natural language descriptions to aid developers' understanding, but such descriptive information is absent in compiler bugs, complicating diagnosis and repair:

- Compiler *crash* bugs (Figure 1b), come with a test case `@test()` (known as a *reproducer* in compiler engineering) and a stack trace that pinpoints where the compiler crashes while compiling the reproducer.
- Compiler *miscompilation* bugs (Figure 1c), offer a reproducer `@src()` and occasionally an input-output pair (known as a *counterexample*) that illustrates how the compiler incorrectly compiles the reproducer into faulty code `@tgt()`, leading to wrong output given the input.
- Other compiler bugs are even more intricate, typically presenting only a reproducer: *slow compilations* and *compiler hangs* indicate compilers unexpectedly take a long time or forever to compile a reproducer; *missed optimizations* denote compilers failing to apply certain beneficial code transformations on a reproducer.

Furthermore, understanding compilers requires specialized tooling and expertise rarely used in general software development, including lexing/parsing, type systems, IR design/optimization, and code generation, which typically take years for human engineers to master. While existing platforms such as SWE-bench (Jimenez et al., 2024) and SWE-agent (Yang et al., 2024b) effectively connect LLMs to standard bash tools for general software engineering tasks, they exhibit limited efficacy in compiler engineering.

^{*}Equal contribution ¹Southern University of Science and Technology ²ETH Zurich ³The Chinese University of Hong Kong. Correspondence to: Shaohua Li <shaohuali@cuhk.edu.hk>, Yuqun Zhang <zhangyq@sustech.edu.cn>.

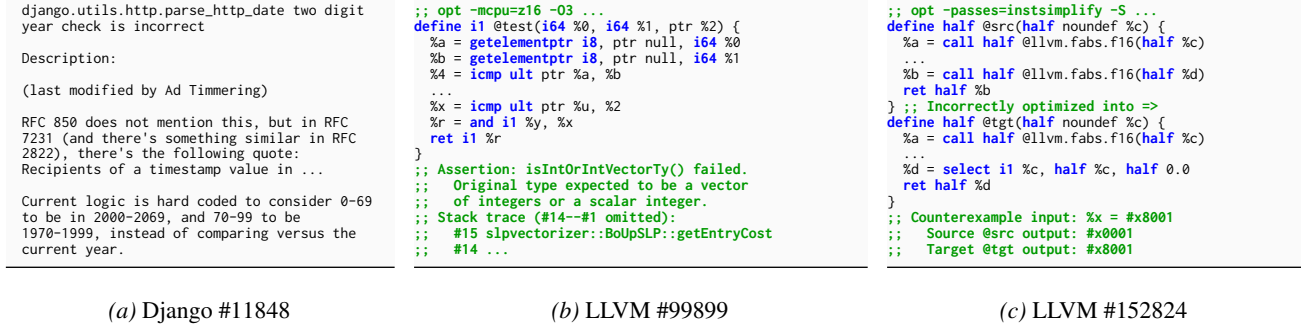


Figure 1. **Compiler issues, lacking descriptive information, are challenging to diagnose and repair.** This compares two LLVM issues (crash and miscompilation) and a common software issue from Django; all issues are simplified for brevity.

To explore the capabilities of LLMs in the context of resolving compiler issues, we put our focus on LLVM, one of the most widely used compilers powering languages from traditional C/C++/Rust to modern Triton/Mojo. At present, we target its middle end, which transforms an LLVM IR program into an optimized version via various optimization components. We prioritize this target due to the expressive, well-defined LLVM IR (LLVM, 2026b) and the complexity of its numerous, target-independent, and error-prone optimizations (LLVM, 2026c; Sun et al., 2016).

11vm-harness. We introduce 11vm-harness, the first specialized harness designed to assist autonomous agents in understanding and fixing compiler bugs. 11vm-harness is divided into three parts (Section 2):

- A set of LLVM-specific tools for building, reproducing, exploring, debugging, and testing LLVM. These tools provide an agent-friendly interface, eliminating unnecessary technical details and allowing agents to focus on the core aspects of bug localization and repair.
- A benchmark called 11vm-bench consisting of 334 reproducible LLVM middle-end bugs, each accompanied by around 1.4 reproducers and 722 regression tests. They are separated into three splits based on difficulty: easy (76.3%), medium (13.2%), and hard (10.5%). The benchmark focuses on the two most common types of compiler bugs: crashes and miscompilations, as performance issues are often less prioritized by LLVM developers. Additionally, we maintain 11vm-bench live, a continuously updated 11vm-bench subset including only issues from the past year.
- A mini agent called 11vm-autofix-mini tailored for fixing LLVM middle-end bugs. Unlike general, state-of-the-art auto-fixing agents such as mini-SWE-agent (SWE-agent, 2024; Yang et al., 2024b), ours draws on our real-world experience and domain knowledge in repairing and maintaining LLVM.

We organized it into four stages: “Setup → Reason → Generate → Validate”, where the reasoning stage and the generation stage construct their agentic loops leveraging LLVM-specific tools from 11vm-harness. 11vm-autofix-mini serves as a demonstration of the harness and as a more promising baseline for future LLVM-specific auto-fixing agents.

Leveraging 11vm-harness, we conduct a systematic evaluation (Sections 4 and 5) of five frontier models to explore and understand their ability to resolve LLVM middle-end issues in 11vm-bench live. The evaluated models include GPT 5, Gemini 2.5 Pro, DeepSeek V3.2, and Qwen 3 Max.

Basic findings from benchmarking experiment. We integrate the state-of-the-art agent mini-SWE-agent, which is designed to bridge the gap between LLMs and common software engineering tasks, with additional 11vm-harness tools to establish a baseline agent. In our experiments with mini-SWE-agent, we find that frontier models struggle with addressing LLVM middle-end bugs in 11vm-harness, despite their demonstrated success with general software bugs in SWE-bench Verified. Specifically, we observe a decline of over 35% (averaging 60%) in resolution rates when replacing SWE-bench Verified with 11vm-bench live. The best-performing model, DeepSeek V3.2, can resolve only 38% of issues—43%, 32%, and 17% of easy, medium, and hard, respectively—compared to its 60% score in SWE-bench Verified (SWE-bench, 2024).

We also find that 11vm-harness offers critical support. On the one hand, removing 11vm-harness tools from mini-SWE-agent results in an additional 38% decrease in resolution rate for its best-performing model, DeepSeek V3.2. On the other hand, our tailored agent 11vm-autofix-mini outperforms the enhanced mini-SWE-agent by 22% per model. With our agent, the best-performing model is GPT 5, which resolves 52% of issues, with 59%, 35%, and 21% of the easy, medium, and hard split, respectively. This highlights the usefulness of

llvm-harness and the necessity of developing compiler-specific harnesses and agents.

More findings from expert review study. To better understand the *true* capabilities of frontier models, we recruit active LLVM developers and maintainers to review all agent-generated patches. Specifically, experts assess whether each agent-generated, benchmark-passing patch is a valid fix for the corresponding bug. The findings are twofold.

First, `llvm-autofix-mini` produces 38% more valid patches than `mini-SWE-agent`, with `llvm-autofix-mini`’s best-performing model, GPT 5, outperforming `mini-SWE-agent`’s top model DeepSeek V3.2 by 40%. Additionally, the percentage of valid patches among benchmark-passing patches is 24% higher for `llvm-autofix-mini` compared to `mini-SWE-agent`, further highlighting the utility of `llvm-harness`.

Second, we find that *truly resolving* LLVM middle-end issues is exceptionally difficult: the true capability of frontier models with both agents is consistently below 21%. In the hard split, only `llvm-autofix-mini` (with GPT 5) manages to resolve a single issue. To investigate why the remaining patches are invalid, we ask experts to document the cause of each invalid patch when reviewing. This results in several actionable directions for future work (Section 7).

Contributions. Our primary contribution is the specialized harness `llvm-harness` and the accompanying `llvm-bench` benchmark, which are pivotal for enabling various agents to work with LLVM effectively. `llvm-autofix-mini` is designed as a demonstration to show `llvm-harness`’ necessity and utility, and to serve as a baseline for future LLVM auto-fixing agents. As a secondary contribution, we conduct a systematic evaluation of frontier models, which uncovers insightful findings and open challenges for future work to address when adopting LLMs to resolve LLVM and compiler bugs. We plan to publicly release `llvm-harness` upon the paper’s publication. They are currently included in the supplementary material.

2. The `llvm-harness` Harness

`llvm-harness` is an off-the-shelf agent harness designed to assist auto-fixing agents in understanding and repairing LLVM middle-end bugs, with the goal of covering all types of LLVM bugs. The task is to generate a patch that resolves the bug and passes all relevant LLVM IR tests, using one or more reproducers that faithfully replicate the bug. In this section, we elaborate on the implementation details of the harness tooling, the benchmark, and the mini agent.

2.1. The Harness Tooling

We wrap common LLVM tasks into agent-accessible tools so that agents can focus on bug repair while the harness handles environment control and validation. Tools belong to the following categories; detailed descriptions are provided in Sections A.2 and A.3 of our supplementary material’s `appendix.pdf`.

Setup & Build. The harness handles build configurations and provides tools for building LLVM with a specific commit and/or target, for example, building for x86 with debug information and assertion checks. The agent can therefore avoid the task of building large systems, a common challenge for autonomous agents (Hu et al., 2025).

Reproduce & Cause. The harness is capable of validating whether a given bug can be triggered faithfully and providing direct causes of reproducible bugs. It sets up LLVM with the bug-containing version, builds it, and runs the reproducer with `opt`, i.e., the LLVM middle-end optimizer. For crash bugs like Figure 1b, it checks whether the crash occurs and provides the stack trace to agents, with uninteresting frames eliminated. For miscompilations like Figure 1c, it ensures `opt` exits normally and validates LLVM’s optimization by applying `alive-tv` (Lopes et al., 2015; 2021), i.e., LLVM’s middle-end translation validator; this will output a counterexample demonstrating the bug if successful.

Explore & Debug. The harness offers search tools for exploring LLVM’s static and dynamic internals. Static information includes commit-specific code, documentation, and LLVM IR’s specification (LLVM, 2026b) through `grep` and `find`-like tools. Dynamic information enables pausing LLVM and inspecting its intermediate states, for example, reading variables and evaluating expressions at a breakpoint, with respect to a reproducer through the `gdb` debugger.

Edit & Patch. With `llvm-harness`, agents can edit LLVM, preview and revert edits, and submit edits to the harness. When necessary, `llvm-harness` checks agents’ edits to prevent irrelevant code changes, such as changes in the frontend, backend, or other tools under the LLVM umbrella.

Test & Validate. The correctness of the submitted patch can be validated, and feedback can be provided. `llvm-harness` will rebuild LLVM and execute relevant tests, typically including reproducers themselves and regression tests of the associated middle-end component such as `InstCombine`. Extensive testing involving regression tests from other components such as `SLPVectorizer` and differential testing of the IR post-optimization (if the golden patch is accessible) are also available. This validation is supported by various compiler-specific engineering tools, such as `opt`, `alive-tv`, and `llvm-lit` (or `FileCheck`). We do not include random testing tools such as `csmith` (Yang

et al., 2011) and reify (Chopra et al., 2026) because they struggle to detect bugs within merely a few hours. Submitted patches can be validated online (i.e., during agent execution) or offline (i.e., after agent execution).

Benchmark & Evaluate. Based on the above tooling, llvm-harness provides scripts to automatically benchmark and evaluate an agent with our benchmark llvm-bench. These scripts output the validated patches (or errors for failed runs), cost and overhead, and trajectories.

2.2. The llvm-bench Benchmark

We build a benchmark to explore and understand the capabilities of frontier models or auto-fixing agents.

Construction. We construct llvm-bench *automatically* by leveraging the llvm-harness tooling. The construction process consists of three stages: (1) issue collection, (2) reproducer validation, and (3) golden patch validation.

Stage I: Issue collection. We collect LLVM’s fixed issues on GitHub and the corresponding commits. For each issue, we include the following details.

- *Type:* Either miscompilation or crash in the middle-end, indicated by the developer-annotated labels. Issues lacking relevant labels or bearing labels such as wontfix, duplicate, and invalid are excluded.
- *Fixing Commit:* The commit that resolves the issue. In LLVM, valid middle-end fixing commits usually alter code in llvm/lib/ or llvm/include/, and usually introduce new tests in llvm/test/ or modify existing ones. Issues are excluded if no such commit exists, is deemed invalid, or reverts LLVM to a previous commit.
- *Reproducers:* One or more LLVM IR files and associated opt command. These LLVM IR programs are extracted using llvm-extract from the fixing commit, based on code changes relevant to tests in llvm/test/.
- *Golden Patch:* Code changes in llvm/lib/ or llvm/include/, as provided in the fixing commit. Stage III will validate the correctness of this patch.
- *Base Commit:* The commit able to reproducing the issue. This is the parent commit of the fixing commit. If the parent commit is not buildable, we trace git history of main branch to identify the most recent buildable commit; we will update the golden patch and reproducers accordingly. This commit will be validated in Stage II.
- *Metadata:* This includes the issue’s title, description, timestamps, additional labels, and developer discussions. Although this information should not be utilized by auto-fixing agents since it may provide hints on bug fixes, it is invaluable for understanding the issue.

Table 1. llvm-bench includes 334 reproducible issues. “#C/#M” is the number of crashes/miscompilations; “#Comp” the number of affected components. “#Rep” and “#Reg” are the average number of reproducers and component-specific regression tests per issue. “Fixes” report the average edited lines, functions, and files per golden patch.

Name	Split	#Bugs (#C/#M)	#Comp	Tests		Fixes		
				#Rep	#Reg	#Lines	#Funcs	#Files
full	full	334 (222/112)	64	1.4	722.2	17.1	1.6	1.2
	easy	255 (176/79)	48	1.4	734.1	9.1	1.0	1.0
	medium	44 (26/18)	18	1.4	626.5	38.8	2.6	1.0
	hard	35 (20/15)	25	1.7	756.6	47.6	4.9	2.7
live	full	229 (160/69)	43	1.5	743.7	18.5	1.7	1.2
	easy	172 (124/48)	31	1.5	769.8	9.0	1.0	1.0
	medium	34 (21/13)	13	1.4	646.7	41.6	2.6	1.0
	hard	23 (15/8)	20	1.8	691.7	55.7	5.1	2.8

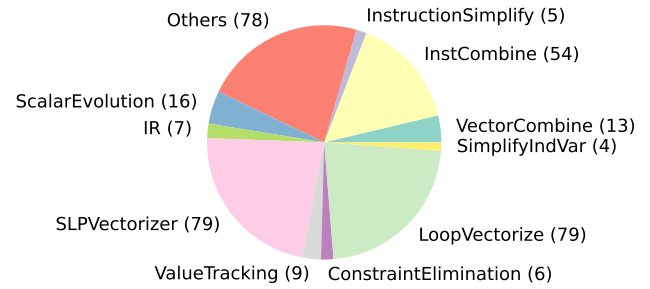


Figure 2. Distribution of Affected Components in llvm-bench

Note that when multiple issues are closed by the same commit, only the oldest issue is retained, because recent issues typically refer to cases of duplication that lack appropriate duplicate labels due to LLVM developers’ oversight.

Stage II: Reproducer validation. We validate whether the extracted reproducers can be reproduced on the base commit leveraging tools for **Reproduce & Cause** (Section 2.1). Issues that cannot be reproduced are excluded.

Stage III: Golden patch validation. We validate whether the golden patch is valid by executing llvm-harness tools for **Test & Validate** (Section 2.1) on the base commit. Issues with the golden patch that cannot be validated are excluded.

Statistics. As of August 26, 2025 when we started benchmark construction, we successfully collected 334 issues starting from January 1, 2024.¹ This includes 222 crashes and 112 miscompilations. An example crash issue is presented in Section C.1 in our supplementary material’s appendix. The upper half of Table 1 with “Name: full” displays the statistics.

¹We did not consider issues prior to this date as old issues might be included in the recent LLMs’ training data. Moreover, llvm-harness is continually updated: When we submitted this paper, we had extended it to 474 reproducible issues.

Affected components. These issues directly affect 64 out of 184² middle-end components in LLVM, with 176 components affected indirectly. LoopVectorizer, SLPVectorizer, and InstCombine—three components known to be the most erroneous (Yang et al., 2011; Livinskii et al., 2020; 2023; Li et al., 2024)—are affected by the greatest number of issues. Figure 2 presents a breakdown; the full list is shown in Section A.1 of our supplementary material’s appendix. These components span 937 C++ files and 676,308 lines of LLVM code. The benchmark lacks issues from other components, as these issues predate January 1, 2024.

Reproducers & tests. On average, each issue is coupled with 1.41 reproducers, 722 component-specific regression tests, and more than 10,000 regression tests from other components. Reproducers and regression tests are minimal LLVM IR programs reduced by automatic tools and LLVM developers. Each reproducer has 49.74 LoC on average.

Fixes & patches. Resolving these LLVM issues is challenging. When LLVM developers initially submitted these patches for code review, fewer than 50% were accepted without further bug fixes; the remainder required corrections based on reviewers’ feedback and subsequent resubmission. In the end, every developer-provided golden patch involves around 17.06 edited lines, 1.62 functions, and 1.17 files.

Three splits. Based on the difficulty in resolving these issues, we divide llvm-bench into three splits: easy issues (255, 76.3%), which can be resolved by updating a single function; medium issues (44, 13.2%), which require changes to multiple functions within the same file; and hard issues (35, 10.5%), which necessitate changes across multiple files. As the splits become more difficult, code fixes become more complex. For example, the hard split involves edits to around five functions spanning around three files on average.

One subset: llvm-bench live. Evaluating agents across the entire llvm-bench can be both time-consuming and costly; our experiments indicate that even the best-performing model requires at least 15 minutes and 1.3 million tokens to fix an issue. On the other hand, old issues may become outdated or less useful for the most recent LLMs. Therefore, we create a *continually* updated live subset that always includes llvm-bench issues from the *latest year*. At present, as shown in the lower half of Table 1, it has 229 issues starting from August 26, 2024, spanning 43 directly affected components and 175 indirectly affected components. Given the evolving nature of compilers, such as LLVM, an agent’s performance on llvm-bench live can reflect its compatibility and effectiveness with the latest compiler versions. Therefore, this subset aims to mitigate the data leakage problem, a common challenge for benchmarking LLMs (Deng et al., 2024; OpenAI, 2026).

²Counted based on LLVM 21.1.0’s -O3 optimization pipeline.

2.3. The llvm-autofix-mini Agent

To demonstrate the practicality and benefits of our harness, we design a mini agent that takes advantage of the tooling. Unlike typical auto-fixing agents (Yang et al., 2024b; Mathai et al., 2025) that resolve software bugs based solely on static information from bug reports and software repositories, llvm-autofix-mini additionally utilizes LLVM’s runtime information obtained through the reproducer. We structure it into a four-stage agent: “Setup → Reason → Generate → Validate”. The first two stages perform root cause analysis to explore information beneficial for the third stage; the final stage handles post-generation, offline patch validation. If the agent unexpectedly exits during the first three stages (e.g., due to reaching the token limit) or if the patch submitted to the final stage is invalid, it results in a failure. Otherwise, a validated patch, which we call a *passed patch*, is output.

Stage I: Setup. llvm-autofix-mini configures the environment using llvm-harness by validating the reproducibility of the reproducer, launching LLVM under gdb with the reproducer, and pausing it at specific breakpoints. For crash bugs, it pauses before the crashing function; for miscompilation bugs, before the first transformation. These breakpoints are chosen because, based on our experience in fixing LLVM bugs, minimal reproducers usually indicate that the root cause is in the neighborhood of these points. This pause allows our agent to examine LLVM’s internal state and detect errors through dynamic debugging. This stage also infers the erroneous component, which is then passed to the LLM in the next *Reason* stage and used for component-specific online testing.

Stage II: Reason. Our agent enters a Re-Act (Yao et al., 2023) agentic loop to deduce the root cause. Within this loop, it has access to exploration-only tools. It may invoke tools to jump to other frames such as `debug(cmd=frame 3)`, inspect local states such as `eval(expr=WidePhi)`, or review code or docs such as `code(func=llvm::VPTransformState::get)` and `docs(func=llvm::VPTransformState::get)`. The loop concludes once the agent successfully identifies the root cause.

Stage III: Generate. Based on the root cause determined in the previous stage, it initiates a new Re-Act agentic loop for patch synthesis. This loop augments the agent with editing and testing capabilities. It may use editing tools to amend or reset LLVM such as `edit(file=.../SLPVectorizer,text=..., replace=...)` and `reset()`, or testing tools for online testing such as `test()`, with the latter offering feedback on failures. The loop ends when online testing is successful.

Stage IV: Validate. This stage consists of offline testing to assess the correctness of the patch generated in the last

stage. It outputs the patch for successful validation.

3. Benchmarking Experiment - Setup

We evaluate frontier LLMs’ performance on `llvm-bench live` using `llvm-harness`. We avoid `llvm-bench` in order to maintain a fair, up-to-date evaluation.

Models. We select four frontier models released in late 2025 when we started our evaluation: GPT 5, Gemini 2.5 Pro, DeepSeek V3.2, and Qwen 3 Max. We further select GPT 4o as a baseline model, released about a year prior to the selected frontier models and before the oldest issue in `llvm-bench live`. All models use the latest versions as of October 1, 2025.³

Agents. SWE-bench Verified (OpenAI, 2024; Jimenez et al., 2024) (or SWEV) and mini-SWE-agent (SWE-agent, 2024; Yang et al., 2024b) are the *de facto* benchmark and agent for measuring frontier models’ agentic capability on common software engineering tasks. As far as we know, all frontier models, including those from OpenAI, Google, DeepSeek, and Alibaba, report agentic results with them. We therefore adopt mini-SWE-agent to `llvm-harness` as the canonical baseline agent and compare it against our agent `llvm-autofix-mini`.

mini-SWE-agent is given access to all bash tools available in Ubuntu (e.g., `grep`, `sed`) and LLVM (e.g., `opt`, `lli`, `llvm-lit`), following SWE-bench’s Bash Only mode. We further enhance mini-SWE-agent with reproducers and the following `llvm-harness`-specific resources: (1) The bug reproduction process from `llvm-autofix-mini`’s Stage I (Section 2.3), which identifies erroneous components, the root cause, the stack trace for crash bugs, and a counterexample for miscompilation bugs; (2) The `test` tool for building and testing LLVM, along with relevant feedback; (3) The post-generation validation process from `llvm-autofix-mini`’s Stage IV (Section 2.3). In contrast, `llvm-autofix-mini` has access to a mini subset of `llvm-harness` tools: `find`, `grep`, `list`, `read`, `edit`, `code`, `docs`, `langref`, `debug`, `eval`, `reset`, `preview`, and `test`.

We do not use larger, production-oriented agents such as Gemini CLI (Gemini, 2025) and Codex (OpenAI, 2025). These agents orchestrate many tools and heuristics, which can *mask* the underlying model behavior: they may fix bugs primarily through extensive scaffolding rather than the model’s own problem-solving ability. However, both mini-SWE-agent and `llvm-autofix-mini` are designed to

³An exception is DeepSeek V3.2, released on December 1, 2025. Our initial experiments (late November to early December) used DeepSeek V3.2 Exp, the then-current version. A silent update to V3.2 on December 1 caused mixed results, with some data from V3.2 Exp and some from V3.2, so we repeated the experiment using V3.2 only.

Table 2. `llvm-bench live` is a challenging benchmark for current LLMs. Fixing LLVM issues is more difficult than fixing common software issues with mini-SWE-agent. “SWEV” denotes SWE-bench Verified; the data are copied verbatim from the SWE-bench (Bash Only) leaderboard or the respective paper. “full”, “easy”, “medium”, and “hard” are splits.

Model	% Passed					\$Cost	
	SWEV	full	easy	medium	hard	SWEV	full
GPT 4o	21.6	8.3 (-61.6%)	9.9	5.9	0.0	1.53	1.59
GPT 5	65.0	21.0 (-67.8%)	23.3	17.6	8.7	0.28	0.74
Gemini 2.5 Pro	53.6	9.2 (-82.9%)	10.5	5.9	4.3	0.29	0.81
Qwen 3 Max	69.6	24.4 (-64.9%)	29.1	17.6	0.0	–	4.32
DeepSeek V3.2	60.0	38.9 (-35.2%)	43.0	32.4	17.4	0.03	0.08
Mean	50.1	17.2 (-60.2%)	19.8	12.8	8.7	0.25	0.80

be *mini*, making them suitable for isolating and comparing the models’ inherent agentic ability. Following LLVM’s policy on AI tools (LLVM, 2026a), we plan to integrate these agents into `llvm-harness` in the future for real-world LLVM repair and maintenance. The system prompts for both agents are in Appendix’s Sections C.2 to C.4.

Parameters. We use the following settings for both agents: temperature 0, context window 64,000 tokens, default reasoning effort, chat limit 500 rounds, token limit 5 million tokens, edit limit 25 calls, and online test limit 25 calls.

Research questions. We assess whether the agent can produce patches that successfully resolve bugs. As mentioned earlier, a patch is considered a *passed patch* if it passes all applied online and offline tests, and therefore each resolved issue is associated with one passed patch. The results are presented as the rate of producing passed patches (“%Passed = #PassedPatches / #Issues”) according to the pass@1 metric. We then analyze results toward three research questions:

Benchmark and model performance (Section 4.1). Using mini-SWE-agent, we compare each model’s performance on `llvm-bench live` with its performance on SWEV. This helps understand whether `llvm-bench` is more challenging.

Effectiveness of the harness (Section 4.2). To evaluate the effectiveness of `llvm-harness`, we select the best-performing model from the previous experiment and rerun it with `llvm-harness`-specific resources excluded.

Baseline comparison and common failures (Section 4.3). We compare the performance of each model with our enhanced mini-SWE-agent and `llvm-autofix-mini`. This assesses whether `llvm-autofix-mini` is a more promising baseline for LLVM middle-end issues. For unresolved issues, we categorize their common failures.

Table 3. llvm-harness improves mini-SWE-agent’s performance significantly. With llvm-harness, mini-SWE-agent (with DeepSeek V3.2) achieves a 62% improvement in %Passed when addressing LLVM middle-end bugs.

Harness	full	Difficulty			Bug Type	
		easy	medium	hard	crash	miscomp
×	24.0	26.2	17.6	17.4	28.1	14.5
✓	38.9	43.0	32.4	17.4	41.9	31.9
Improv.	1.6×	1.6×	1.8×	1.0×	1.5×	2.2×

4. Benchmarking Experiment - Results

4.1. Benchmark and Model Performance

We first evaluate the capabilities of the widely used mini-SWE-agent in resolving llvm-bench live issues and compare its performance with SWEV. In this experiment, we follow established academic and industry practice (Xia et al., 2025; Chen et al., 2025) to report SWEV leaderboard numbers instead of running the benchmark in our own environment. We use this cross-benchmark comparison to *illustrate* (rather than rigorously quantify) the difficulty of llvm-bench and compiler bugs.

Table 2 shows that mini-SWE-agent faces greater challenges when addressing LLVM middle-end issues in llvm-bench compared to common software bugs in SWEV. Even with a budget of five million tokens, which exceeds the typical mini-SWE-agent settings for SWEV, the performance of these models still declines by over 35% (average: 60.2%) when SWEV is replaced with llvm-bench live. Current LLMs’ repair capabilities against LLVM issues, although all improved over GPT 4o, are also suboptimal. The best-performing model is DeepSeek V3.2, yet its performance (89/229) is less than 39%; for Gemini 2.5 Pro, only 21 out of 229 (less than 10%) issues are resolved successfully. GPT 5 and Qwen 3 Max fall in between (less than 25%). These results lag behind their performance on SWEV.

By difficulty. The situation becomes worse as the benchmark difficulty increases across splits. Generally, the resolution rate for the medium and hard splits is consistently below the average for the full set, but that for the easy split is higher. The baseline GPT 4o model can handle fewer than 9% of the issues, while for the issues in the hard split, it is unable to handle any of them. The four frontier models can partially handle easy LLVM bugs, underperform in medium (~30% worse), while struggling or even failing in hard (further ~60% worse). It is interesting to note that Qwen 3 Max’s performance declines sharply when moving to hard, whereas the other models degrade more smoothly.

Table 4. llvm-autofix-mini is a more promising baseline in repairing LLVM issues than mini-SWE-agent. Detailed data regarding the cost and overhead is presented in Table 5.

Model	mini-SWE-agent		llvm-autofix-mini	
	%Passed	\$Cost	%Passed	\$Cost
GPT 4o	8.3	1.59	12.2 (1.5×	2.18
GPT 5	21.0	0.74	51.5 (2.5×	0.59
Gemini 2.5 Pro	9.2	0.81	14.4 (1.6×	1.16
Qwen 3 Max	24.4	4.32	35.8 (1.5×	5.67
DeepSeek V3.2	38.9	0.08	10.5 (0.3×	0.15
Mean	17.2	0.80	20.3 (1.2×	1.05

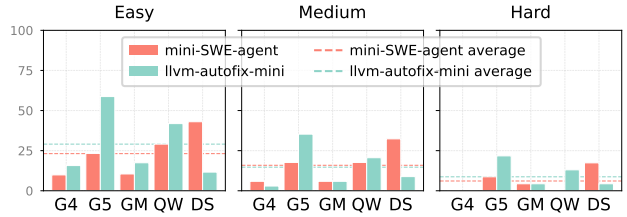


Figure 3. llvm-autofix-mini outperforms mini-SWE-agent at each split, but as splits increase in difficulty, frontier models tend to struggle or fail. The Y-axis is “%Passed”. G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2.

4.2. Effectiveness of the Harness

To evaluate the effectiveness of llvm-harness, we rerun the previous experiment after excluding llvm-harness. For this rerun, we use only mini-SWE-agent with DeepSeek V3.2, as llvm-autofix-mini is tailored specifically for llvm-harness, and DeepSeek V3.2 demonstrates the best performance in mini-SWE-agent. Specifically for mini-SWE-agent, we remove (1) the bug reproduction process, (2) the `test` tool, and (3) the post-validation process, as detailed in Section 3. While alive-tv and LLVM bash tools such as `opt` are included in llvm-harness, we do not consider them to be our contributions; they remain accessible to mini-SWE-agent. We compare with the previous experiment to assess its effectiveness.

Table 3 shows that llvm-harness plays a critical role in assisting mini-SWE-agent to fix LLVM middle-end bugs: removing it reduces performance by 38%, bringing %Passed down to 24.0%; conversely, the inclusion of it leads to a significant 62% improvement in mini-SWE-agent’s performance. The easy and medium splits show performance gains of 64% and 85%, respectively. However, no improvement is observed in the hard split, likely due to the inherent difficulty of these cases reaching the limit of mini-SWE-agent in our experimental setup. The improvement in addressing miscompilations is more pronounced compared to fixing crashes in the middle end.

4.3. Baseline Comparison and Common Failures

Finally, we compare the performance of `llvm-autofix-mini` with `llvm-harness-enhanced-mini-SWE-agent`. As shown in Table 4, `llvm-autofix-mini`’s performance is generally $1.22\times$ of `mini-SWE-agent`’s with affordable cost. A statistical test confirming `llvm-autofix-mini`’s significant improvement is presented in Section B.1 of our appendix. The greatest improvement occurs with GPT 5 (+145%), whereas DeepSeek V3.2 is an exception (-73%): when executed with `llvm-autofix-mini`, it frequently fails to adhere to our tool-calling format, whereas such failures are seldom seen with `mini-SWE-agent`; this results in a large (>85%) portion of failures exceeding the token budget.⁴ The improvement for GPT 4o, Gemini 2.5 Pro, and Qwen 3 Max is 47%, 57%, and 47%, respectively. For all agent-model pairs, GPT 5 successfully resolves the largest number of issues (118/229, 51.5%) when executed with `llvm-autofix-mini`, followed by DeepSeek V3.2 with `mini-SWE-agent` (89/229, 38.9%) and Qwen 3 Max with `llvm-autofix-mini` (82/229, 35.8%). The bug repair capability of the four frontier models is generally better than the baseline model: improved by around 16% when executed with `llvm-autofix-mini`, compared with GPT 4o. The improvement of the GPT, Qwen, and DeepSeek models is more marked than that of the Gemini model. The same trend, i.e., `llvm-autofix-mini` outperforming `mini-SWE-agent`, is consistently observed across different difficulty splits (Figure 3) and bug types (Figure 4).

By difficulty. The resolution rate decreases as splits become more difficult for both agents. The hard split poses a significant challenge (Figure 3): Every selected model can fix certain bugs in the easy or medium split, but in the hard split, `llvm-autofix-mini` cannot drive GPT 4o to repair any issue. In particular, `mini-SWE-agent`’s average resolution rates are 23.2%, 15.8%, and 6.1% for easy, medium, and hard, respectively, whereas the data for `llvm-autofix-mini` are 28.8% (1.8 $\times$), 14.7% (0.9 $\times$), and 7.8% (1.3 $\times$). The degradation for the medium split is primarily due to DeepSeek V3.2 as described earlier. We believe that this indicates the necessity of developing LLVM-specific auto-fixing agents with `llvm-harness`, rather than relying on a general-purpose agent scaffold such as `mini-SWE-agent` which targets common software. `llvm-autofix-mini` serves as a more promising baseline.

By bug type. Figure 4 illustrates both agents’ performance breakdown across different bug types. Generally, miscompilations are more challenging to diagnose and repair than crashes, which aligns with our experience.

⁴Note that this is a commonly encountered, DeepSeek V3.2-specific issue reported in other settings as well (vLLM, 2025a;b; SGLang, 2025; 2026), rather than an issue of our format.

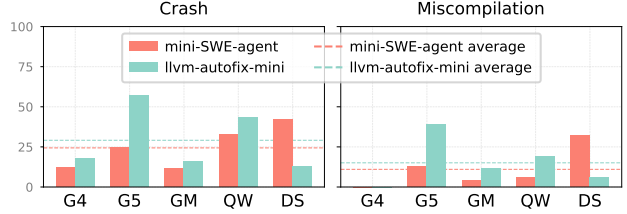


Figure 4. `llvm-autofix-mini` outperforms `mini-SWE-agent` for each bug type, and miscompilations are more challenging than crashes. The Y-axis is “%Passed”. G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2.

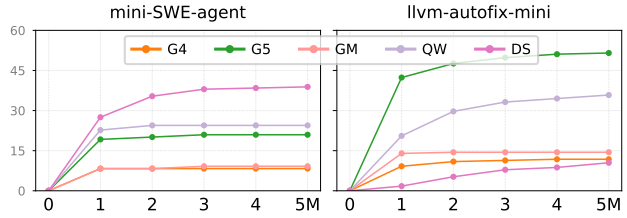


Figure 5. `llvm-autofix-mini` is more robust as token limit increases than `mini-SWE-agent`. The Y-axis is “%Passed”. For `mini-SWE-agent`, nearly all models converge after reaching three million tokens. G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2.

The resolution rate of miscompilations lags approximately 13% and 14% behind crashes for `llvm-autofix-mini` and `mini-SWE-agent`, respectively. The baseline model GPT 4o is unable to resolve any miscompilations, whether using `mini-SWE-agent` or `llvm-autofix-mini`. Among frontier models, GPT 5 and DeepSeek V3.2 are the best-performing models for `llvm-autofix-mini` and `mini-SWE-agent`, respectively, whereas Gemini 2.5 Pro’s capability usually falls below average for either bug type.

With token limit. We investigate whether the agent’s performance improves as token limits increase. Figure 5 displays this trend. `mini-SWE-agent`’s performance appears to converge when the token limit reaches three million. Although `llvm-autofix-mini` shows a similar pattern with GPT 4o and Gemini 2.5 Pro, `llvm-autofix-mini` is more robust: the performance of the other three frontier models increases smoothly. Aside from DeepSeek V3.2, `llvm-autofix-mini` consistently outperforms `mini-SWE-agent` once the token budget exceeds two million and continues to improve until the five-million budget. However, for the one-million token budget, `llvm-autofix-mini` performs slightly worse than Qwen 3 Max due to the additional reasoning stage required before attempting fixes. It is also interesting to note that the performance of DeepSeek V3.2 increases more markedly than that of other models when executed with `llvm-autofix-mini`, and it is expected to surpass GPT 4o as the limit extends

Table 5. Cost and Overhead of `llvm-autofix-mini` and `mini-SWE-agent`. M: Model; T/m: Time/min; #Tot/#In/#Out/#Cache: The total/input/output/cached tokens consumed (measured in millions unless otherwise specified); G4, G5, GM, QW, and DS: GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2.

M	T/m	#Tests	#Rounds	Token Cost (million)					\$Cost
				#Tot	#In	#Out	#Cache	%Cache	
mini-SWE-agent									
G4	7.1	13.0	74.4	2.2	2.2	6.9K	2.2	96.4	1.6
G5	14.6	9.5	66.9	2.6	2.6	28.2K	2.5	94.4	0.7
GM	8.8	7.4	51.7	1.6	1.6	21.3K	1.4	85.8	0.8
QW	18.1	13.1	77.7	2.4	2.4	15.3K	1.1	43.9	4.3
DS	15.1	3.9	80.0	2.3	2.3	17.2K	2.2	97.9	0.1
llvm-autofix-mini									
G4	15.5	10.4	116.5	3.1	3.1	7.7K	3.0	96.3	2.2
G5	16.1	6.4	39.0	1.3	1.3	32.3K	1.2	90.0	0.6
GM	12.5	3.5	53.2	1.5	1.5	21.5K	1.3	83.3	1.2
QW	25.5	8.0	81.4	3.4	3.4	16.8K	1.7	49.3	5.7
DS	21.8	1.3	183.5	4.7	4.7	16.3K	4.6	98.5	0.2

to six million tokens. An explanation for this could be that DeepSeek V3.2 progressively adheres to our tool calling format as more interactions are involved.

On the other hand, the performance of these models does *not* notably improve with an increased number of tokens. We propose two hypotheses in this paper: (1) The “context rot” problem (Hong et al., 2025) becomes more pronounced as agents operate for extended periods (resulting in increased context). (2) The complexity of compiler bugs makes it increasingly infeasible to resolve the remaining issues within a reasonable token limit.

Cost and overhead. Fixing LLVM issues necessitates more cost and runtime overhead compared with common software issues in SWEV (Table 2). The average overhead (Table 5) for resolving an issue using the four frontier models with `mini-SWE-agent` is 14.1 minutes, ranging from 7.1 to 18.1 minutes. While `llvm-autofix-mini` requires an additional 4.8 minutes on average, this overhead remains practical. Additionally, the costs for both agents are highly affordable (maximum \$6/issue); for their best-performing models, the cost is as low as \$0.1 and \$0.6 per issue, respectively. We also observe that `llvm-autofix-mini` typically engages in more rounds of reasoning and information gathering, while requiring fewer `test` calls compared to `mini-SWE-agent`. This suggests that our design enables models to make more informed and efficient decisions. Our agent is as cache-friendly as `mini-SWE-agent`, enabling it to fully leverage the token budget.

Common LLM failures. We group failures by LLMs in Figure 6. GPT 4o typically fails due to context limitation (CtxtLimit): it frequently calls tools such as `read` whose output is long, exceeding the 64K-token context window. The four frontier models rarely have such fail-

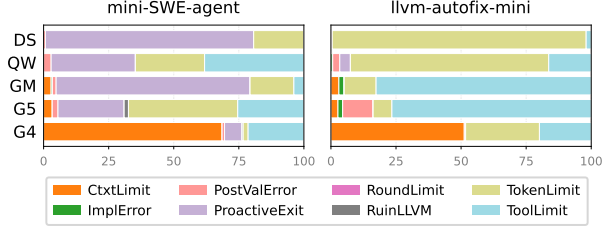


Figure 6. Failure distribution of unresolved issues. G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2, respectively.

Table 6. Current LLMs lack satisfactory, true capability in fixing LLVM middle-end issues: Despite passing all tests, fewer than 42% of passed patches are valid. “%Valid = #ValidPatches / #Issues” is the percentage of valid patches.

Model	mini-SWE-agent		llvm-autofix-mini	
	%Passed	%Valid	%Passed	%Valid
GPT 4o	8.3	1.7 (21.0%)	12.2	3.9 (32.1%)
GPT 5	21.0	6.6 (31.3%)	51.5	20.1 (39.0%)
Gemini 2.5 Pro	9.2	2.2 (23.8%)	14.4	5.2 (36.4%)
Qwen 3 Max	24.4	8.7 (35.7%)	35.8	13.1 (36.6%)
DeepSeek V3.2	38.9	14.4 (37.1%)	10.5	4.4 (41.7%)
Mean	20.3	6.7 (29.8%)	24.9	9.3 (37.1%)

ures, possibly due to their stronger understanding capability. Instead, their failure symptoms vary depending on the underlying agent. For `llvm-autofix-mini`, the majority of failures are TokenLimit and ToolLimit, i.e., reaching the limit of the token budget (5M tokens) or tool budget (25 `edits` or 25 `tests`). If given a larger budget, we expect `llvm-autofix-mini` to continue executing. Although TokenLimit and ToolLimit account for nearly half as well, a large number of `mini-SWE-agent` failures are ProactiveExit where the agent proactively exits the repair process, declaring that it cannot fix the issue. There are cases where `mini-SWE-agent` corrupts the LLVM repository (RuinLLVM), leading to crashes during patch validation; such failures do not occur with `llvm-autofix-mini`. Post validation failures (PostValError) take up to 12%. Two bugs in `llvm-autofix-mini` caused several ImplErrors. We did not observe RoundLimit errors.

5. Expert Review Study

Although passing all tests, passed patches may still be invalid due to the limitation of dynamic testing. Therefore, to better understand the *true* capability of frontier models, we employ an expert, who is an active LLVM developer and maintainer, to review every passed patch from `mini-SWE-agent` and `llvm-autofix-mini`. Although involving more experts would reduce bias and strengthen the study, review capacity is a scarce resource in the LLVM community. The project itself therefore adopts a single-reviewer

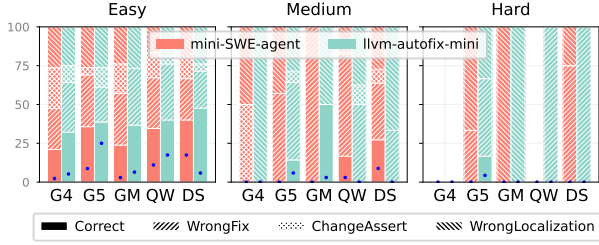


Figure 7. As splits increase in difficulty, frontier models’ true capability degrades fast: No models can handle the hard split except GPT 5 (llvm-autofix-mini). G4, G5, GM, QW, and DS are short for GPT 4o, GPT 5, Gemini 2.5 Pro, Qwen 3 Max, and DeepSeek V3.2, respectively. Blue bullets (•) represent the rate of truly resolved issues, i.e., “%Valid” in Table 6.

model, where a patch can be committed once one reviewer approves it (LLVM, 2026d). We follow this established practice. The expert labels *valid patches* (semantically sound and review-worthy for LLVM developers) and categorizes common LLM mistakes among invalid ones. That said, in this study, we recognize an issue as being *truly resolved* if the passed patch is labeled valid by the expert reviewer. Table 6 shows the percentage of valid patches after code review (“%Valid = #ValidPatches / #Issues”), as well as their proportion among passed patches (in gray). Our interpretation of the review result is twofold.

First, we observe that llvm-autofix-mini’s advantage becomes even more pronounced after code review: (1) it produces 38% more valid patches than mini-SWE-agent, up from a 22% advantage before code review, and (2) the percentage of valid patches among benchmark-passing patches is 24% higher compared to mini-SWE-agent. Our best-performing model, GPT 5, surpasses mini-SWE-agent’s best-performing model DeepSeek V3.2 by 40%. Similar trends are also observed in each split (Figure 7) and bug type (Table 7). The expert even found instances where the llvm-autofix-mini-provided patch is better than the golden patch provided by LLVM developers; such an example is present in the appendix (Section C.5).

Second, however, we find that *truly resolving LLVM middle-end issues is exceptionally difficult*. Despite advances over the baseline model GPT 4o, frontier models still do not possess adequate, true capabilities for repairing LLVM middle-end issues. Even for the best-performing agent llvm-autofix-mini (GPT 5), more than 60% of passed patches are invalid, resulting in only 20.1% of issues being truly resolved; all other agent-model pairs are consistently below 15%, where 7 out of 10 agent-models are below 9% (21/229). The results become even worse as the split difficulty increases (Figure 7). Many agent-model pairs (6/10) are unable to output any valid patches for the medium split. In terms of the hard split, only GPT 5 manages to resolve

Table 7. Miscompilations are more challenging than crashes. Valid miscompilation patches are usually fewer than valid crash patches. “miscomp” stands for miscompilation.

Type	Model	mini-SWE-agent		llvm-autofix-mini	
		%Passed	%Valid	%Passed	%Valid
crash	GPT 4o	11.9	2.5 (21.1%)	17.5	5.6 (32.1%)
	GPT 5	24.4	6.9 (28.2%)	56.9	18.1 (31.9%)
	Gemini 2.5 Pro	11.2	2.5 (22.2%)	15.6	6.2 (40.0%)
	Qwen 3 Max	32.5	11.2 (34.6%)	43.1	15.0 (34.8%)
	DeepSeek V3.2	41.9	11.9 (28.4%)	12.5	4.4 (35.0%)
	Mean	21.3	5.6 (26.5%)	24.2	8.4 (34.6%)
miscomp	GPT 4o	0.0	0.0 (0.0%)	0.0	0.0 (0.0%)
	GPT 5	13.0	5.8 (44.5%)	39.1	24.6 (63.0%)
	Gemini 2.5 Pro	4.3	1.4 (33.3%)	11.6	2.9 (25.0%)
	Qwen 3 Max	5.8	2.9 (50.0%)	18.8	8.7 (46.2%)
	DeepSeek V3.2	31.9	20.3 (63.6%)	5.8	4.3 (75.0%)
	Mean	10.1	4.7 (46.6%)	14.9	7.2 (48.3%)

a single issue with llvm-autofix-mini. This %Passed-%Valid gap arises in all LLMs because LLVM tests typically capture specific symptoms, rather than a complete compilation specification. When comparing crashes and miscompilations (Table 7), the rate of valid miscompilation patches is generally lower than that of valid crash patches for both agents, aligning with the observation that miscompilations are more challenging.

Common LLM mistakes. The expert identifies three categories of LLM mistakes (examples in Section C.5):

- **ChangeAssert.** LLMs “cheat” the system by altering the assertion condition to avoid assertion failures (one type of crash bugs). Their erroneous code changes include (1) changing the assertion condition itself, (2) modifying functions directly or indirectly invoked by the assertion condition, and (3) inserting early return statements preceding the assertion.
- **WrongLocalization.** Despite llvm-harness providing llvm-autofix-mini and mini-SWE-agent with the erroneous component, models still struggle to localize bugs within erroneous files and functions, particularly when bugs impact multiple of them. Detailed data regarding bug localization is presented in Section B.2 of the appendix in our supplementary material.
- **WrongFix.** Models generate passed yet faulty patches although with correct localization. However, for LLVM and other compilers, such patches often suffer from several limitations: (1) they bypass the erroneous component by weakening or strengthening the activation condition instead of addressing the issue, (2) they lack generality beyond the specific reproducers and regression tests, and (3) they silently introduce new bugs, such as missed optimizations that inhibit subsequent opti-

mizations. Although this does not affect correctness, it negatively affects the performance of generated code.

6. Discussion

In summary, our benchmarking experiments and expert review study of frontier models highlight a key finding:

Truly resolving LLVM middle-end issues in llvm-bench is exceptionally challenging, where specialized harnesses and tailored agents are required.

In this context, llvm-harness and llvm-autofix-mini lay the foundations for the future; see more in Section 7.

Threats to validity. The first threat concerns our use of a single sample to report pass@1. Due to budget limitations, we cannot run multiple trials. We mitigate this threat by using greedy decoding for each model call, a standard choice in the community. Second, there is a potential threat of data contamination. To examine its influence, we compare the resolution rate of issues that were fixed before and after the release of each model. Our results show that data contamination does not appear to have a substantial impact, especially on llvm-autofix-mini. We include the detailed results of this study in Section B.3. Finally, our expert review findings may render “%Passed” used in the benchmarking experiment less useful. However, we argue that it remains valuable because it is objective, reproducible, and fully automated. It indicates an upper bound of each LLM’s capability and allows sound comparison across LLMs. For example, Table 6 shows that the relative ranking of LLMs’ performance remains unchanged after expert review, for both agents.

Limitations and extensions. Our expert review study in Section 5 implies that llvm-harness is currently limited by LLVM’s regression tests, leading to a %Passed-%Valid gap. These tests are inadequate to systematically validate agent-generated patches, especially in detecting the three categories of LLM mistakes. However, unlike common software, addressing such issues poses unique challenges for compilers; we discuss this later in Section 7.

llvm-harness does not currently consider performance issues related to the middle-end, such as slow compilations, compiler hangs, and missed optimizations, nor does it include frontend and backend bugs. In fact, we have already incorporated compiler hangs by the time of submitting this paper. Extending llvm-harness to the backend requires additional engineering effort, such as integrating backend tooling including llc for backend compilation and optimization, target-dependent arm-tv and riscv-tv for translation validation, and mid-backend cross-validation with llubi and llc. The same applies to various LLVM frontends. Our agent llvm-autofix-mini, however, is end-independent and could be adapted by wiring it to the corresponding

toolsets. We are now working on these extensions.

7. Open Challenges for Future Work

Although outperforming mini-SWE-agent, the expert review study exposes several challenges that warrant further advances to create practical, more effective LLVM auto-fixing agents, along two directions.

Patch generation. llvm-autofix-mini achieves at best a resolution rate of only 51.5%, with an average of 24.9% across settings (Section 4.1), highlighting significant room for improvement. Future work should prioritize the development of more advanced agents on patch generation to consistently achieve higher %Passed scores, particularly for medium and hard splits as well as miscompilation bugs. For instance, many of our failures stem from token/tool limitations (Section 4.3) and incorrect bug localization (Section 5). Enhancing agents’ expertise in LLVM-related tasks should therefore be a primary focus; a promising avenue is to distill knowledge and bug-fixing experiences from LLVM resources and developers into reusable skills. Instead of relying on static workflows (Section 2.3), next-generation agents may benefit from adopting more flexible, autonomous designs that dynamically adapt to varying levels of task difficulty. Compilers’ complexity necessitates large context limits (Section 4.3), but long context can lead to the “context rot” problem (Hong et al., 2025). Designing sophisticated context management mechanisms that maintain concise yet tractable context is also important.

Patch validation. Section 5 shows there exists a %Passed-%Valid gap, underscoring the limitations of relying solely on LLVM tests. It is crucial to develop automated patch validation techniques or agents because relying on experts’ manual review to reduce the gap is impractical. First, LLMs can manipulate logic by altering conditions or inserting shortcuts that bypass essential checks or optimizations. While such bypasses might lead to errors in general software, in the context of compilers, they result in technically *correct yet suboptimal* code, a *unique challenge* in the compiler domain. Patch validation techniques should discover such bypasses, and further detect such missed optimizations. Second, LLMs tend to stop repairs when current tests are passed. Such “oversight” often results in overfitting patches that fail to optimize unseen code, leading to new bugs. Patch validation techniques should be able to handle these patches.

8. Related Work

Compiler testing. Recent research has increasingly leveraged LLMs to enhance compiler validation and verification. Some works integrate LLMs into the fuzzing process to generate random input programs (Xia et al., 2024; Yang et al.,

2024a; Ni & Li, 2025; Munley et al., 2024) and mutation operators (Ou et al., 2025), targeting crash bugs and miscompilation bugs. Other works focus on detecting performance issues such as missed-optimizations (Italiano & Cummins, 2025; Xu et al., 2026). These works demonstrate significant potential, and llvm-harness offers an opportunity to address the bugs that they uncover automatically through frontier models and our agents.

Compiler optimization. Integrating traditional ML techniques and LLMs into the compilation pipeline also shows promise for improving both code size and performance. These works usually target specific compiler components, such as inlining (Trofin et al., 2021; Cummins et al., 2025), register allocation (VenkataKeerthy et al., 2023; Cummins et al., 2025), loop optimization (Taneja et al., 2025), pass scheduling (Kulkarni & Cavazos, 2012), auto-tuning (Ashouri et al., 2018), and super optimization (Wei et al., 2025b;a). In this context, CompilerGym provides a toolkit for applying reinforcement learning to compiler optimization tasks (Cummins et al., 2022). Unlike ours, their focus is on improving the optimizations themselves rather than addressing reported bugs within them.

Other systems tasks. Beyond compilers, LLMs have been applied to other systems tasks. For instance, certain studies focus on Linux kernel crash diagnosis and repair, offering benchmarks and environments to evaluate LLM-based agents (Mathai et al., 2024; 2025; Shehada et al., 2025). Additionally, research has explored LLMs’ ability to synthesize efficient GPU kernels (Ouyang et al., 2025).

9. Conclusion

Despite LLMs’ success in common software, they face challenges with compiler-specific issues. llvm-harness offers essential LLVM-specific tools and a challenging benchmark, promoting the development of specialized agents like llvm-autofix-mini. Our expert reviews also highlight ongoing challenges, including improving model expertise and devising robust patch validation methods.

References

- Ashouri, A. H., Killian, W., Cavazos, J., Palermo, G., and Silvano, C. A survey on compiler autotuning using machine learning. *ACM Comput. Surv.*, 2018.
- Chen, Z., Tang, R., Deng, G., Wu, F., Wu, J., Jiang, Z., Prasanna, V., Cohan, A., and Wang, X. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL ’25, 2025.
- Chopra, K., Li, C., Sotiropoulos, T., and Su, Z. Semantic reification: A new paradigm for random program generation. *Proc. ACM Program. Lang.*, (PLDI), 2026.
- Cummins, C., Wasti, B., Guo, J., Cui, B., Ansel, J., Gomez, S., Jain, S., Liu, J., Teytaud, O., Steiner, B., Tian, Y., and Leather, H. CompilerGym: Robust, performant compiler optimization environments for AI research. In *Proceedings of the 2022 20th IEEE/ACM International Symposium on Code Generation and Optimization*, CGO ’22, 2022.
- Cummins, C., Seeker, V., Grubisic, D., Roziere, B., Gehring, J., Synnaeve, G., and Leather, H. LLM Compiler: Foundation language models for compiler optimization. In *Proceedings of the 2025 34th ACM SIGPLAN International Conference on Compiler Construction*, CC ’25, 2025.
- Deng, C., Zhao, Y., Tang, X., Gerstein, M., and Cohan, A. Benchmark probing: Investigating data leakage in large language models. In *Proceedings of the NeurIPS 2023 Workshop on Backdoors in Deep Learning - The Good, the Bad, and the Ugly*, 2024.
- Gemini. Gemini CLI, 2025. URL <https://github.com/google-gemini/gemini-cli>. Accessed: Mar 17th 2026.
- Hong, K., Troynikov, A., and Huber, J. Context rot: How increasing input tokens impacts LLM performance. Technical report, Chroma, 2025.
- Hu, L., Chen, G., Shang, X., Cheng, S., Wu, B., Li, G., Zhu, X., Zhang, W., and Yu, N. CompileAgent: Automated real-world repo-level compilation with tool-integrated LLM-based agent system. In *Proceedings of the 2025 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL ’25, 2025.
- Italiano, D. and Cummins, C. Finding missed code size optimizations in compilers using large language models. In *Proceedings of the 2025 34th ACM SIGPLAN International Conference on Compiler Construction*, CC ’25, 2025.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. SWE-bench: Can language models resolve real-world github issues? In *Proceedings of the 2024 12th International Conference on Learning Representations*, ICLR ’24, 2024.
- Kulkarni, S. and Cavazos, J. Mitigating the compiler optimization phase-ordering problem using machine learning. In *Proceedings of the 2012 ACM International Conference on Object Oriented Programming Systems Languages and Applications*, OOPSLA ’12, 2012.

- Li, S., Theodoridis, T., and Su, Z. Boosting compiler testing by injecting real-world code. *Proc. ACM Program. Lang.*, (PLDI), 2024.
- Livinskii, V., Babokin, D., and Regehr, J. Random testing for C and C++ compilers with YARPGen. *Proc. ACM Program. Lang.*, (OOPSLA), 2020.
- Livinskii, V., Babokin, D., and Regehr, J. Fuzzing loop optimizations in compilers for C++ and data-parallel languages. *Proc. ACM Program. Lang.*, (PLDI), 2023.
- LLVM. LLVM AI tool use policy, 2026a. URL <https://llvm.org/docs/AIToolPolicy.html>. Accessed: Mar 17th 2026.
- LLVM. LLVM language reference manual, 2026b. URL <https://llvm.org/docs/LangRef.html>. Accessed: Mar 17th 2026.
- LLVM. LLVM’s analysis and transform passes, 2026c. URL <https://llvm.org/docs/Passes.html>. Accessed: Mar 17th 2026.
- LLVM. Llvm code-review policy and practices, 2026d. URL <https://llvm.org/docs/CodeReview.html>. Accessed: Mar 17th 2026.
- Lopes, N. P., Menendez, D., Nagarakatte, S., and Regehr, J. Provably correct peephole optimizations with Alive. In *Proceedings of the 2015 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI ’15, 2015.
- Lopes, N. P., Lee, J., Hur, C.-K., Liu, Z., and Regehr, J. Alive2: Bounded translation validation for LLVM. In *Proceedings of the 2021 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI ’21, 2021.
- Mathai, A., Huang, C., Maniatis, P., Nogikh, A., Ivancic, F., Yang, J., and Ray, B. kGym: A platform and dataset to benchmark large language models on Linux kernel crash resolution. In *Proceedings of the 2024 38th Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, NeurIPS ’24, 2024.
- Mathai, A., Huang, C., Ma, S., Kim, J., Mitchell, H., Nogikh, A., Maniatis, P., Ivančić, F., Yang, J., and Ray, B. CrashFixer: A crash resolution agent for the Linux kernel, 2025. URL <https://arxiv.org/abs/2504.20412>.
- Munley, C., Jarmusch, A., and Chandrasekaran, S. LLM4VV: Developing LLM-driven testsuite for compiler validation. *Future Gener. Comput. Syst.*, 2024.
- Ni, Y. and Li, S. Interleaving large language models for compiler testing. *Proc. ACM Program. Lang.*, (OOPSLA2), 2025.
- OpenAI. Introducing SWE-bench Verified, 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>. Accessed: Mar 17th 2026.
- OpenAI. Codex, 2025. URL <https://github.com/openai/codex>. Accessed: Mar 17th 2026.
- OpenAI. Why swe-bench verified no longer measures frontier coding capabilities, 2026. URL <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>. Accessed: Mar 17th 2026.
- Ou, X., Li, C., Jiang, Y., and Xu, C. The mutators reloaded: Fuzzing compilers with large language model generated mutation operators. In *Proceedings of the 2024 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ASPLOS ’24, 2025.
- Ouyang, A., Guo, S., Arora, S., Zhang, A. L., Hu, W., Re, C., and Mirhoseini, A. KernelBench: Can LLMs write efficient GPU kernels? In *Proceedings of the 2025 42nd International Conference on Machine Learning*, ICML ’25, 2025.
- SGLang. [bug] deepseek-v3.2 occasional malformed tool call output, 2025. URL <https://github.com/sgl-project/sclang/issues/14695>. Accessed: Mar 17th 2026.
- SGLang. [bug] deepseek-v3.2 places tool call content in "content" field instead of "tool_calls" when invoking tools, and finish_reason is "stop" rather than "tool_calls", 2026. URL <https://github.com/sgl-project/sclang/issues/17561>. Accessed: Mar 17th 2026.
- Shehada, K., Wu, Y., Feng, W. D., Iyer, A., Kumfert, G., Ding, Y., and Qian, Z. Rethinking kernel program repair: Benchmarking and enhancing LLMs with RGym. In *Proceedings of the NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling*, 2025.
- Sun, C., Le, V., Zhang, Q., and Su, Z. Toward understanding compiler bugs in gcc and llvm. In *Proceedings of the 2016 25th International Symposium on Software Testing and Analysis*, ISSTA ’16, 2016.
- SWE-agent. mini-swe-agent, 2024. URL <https://github.com/SWE-agent/mini-swe-agent>. Accessed: Mar 17th 2026.
- SWE-bench. Swe-bench leaderboards, 2024. URL <https://www.swebench.com/index.html>. Accessed: Mar 17th 2026.

- Taneja, J., Laird, A., Yan, C., Musuvathi, M., and Lahiri, S. K. LLM-Vectorizer: LLM-based verified loop vectorizer. In *Proceedings of the 2025 23rd ACM/IEEE International Symposium on Code Generation and Optimization*, CGO '25, 2025.
- Trofin, M., Qian, Y., Brevdo, E., Lin, Z., Choromanski, K., and Li, D. MLGO: a machine learning guided compiler optimizations framework, 2021. URL <https://arxiv.org/abs/2101.04808>.
- VenkataKeerthy, S., Jain, S., Kundu, A., Aggarwal, R., Cohen, A., and Upadrastra, R. RL4ReAl: Reinforcement learning for register allocation. In *Proceedings of the 2023 32nd ACM SIGPLAN International Conference on Compiler Construction*, CC '23, 2023.
- vLLM. [bug]: Deepseek v3.2 tool_calls failure, 2025a. URL <https://github.com/vllm-project/vllm/issues/26897>. Accessed: Mar 17th 2026.
- vLLM. [bug]: Frequent tool call parsing failures with deepseek-v3.2, 2025b. URL <https://github.com/vllm-project/vllm/issues/36654>. Accessed: Mar 17th 2026.
- Wei, A., Suresh, T., Tan, H., Xu, Y., Singh, G., Wang, K., and Aiken, A. Improving assembly code performance with large language models via reinforcement learning. In *Proceedings of the NeurIPS 2025 4th Workshop on Deep Learning for Code*, 2025a.
- Wei, A., Suresh, T., Tan, H., Xu, Y., Singh, G., Wang, K., and Aiken, A. SuperCoder: Assembly program super-optimization with large language models, 2025b. URL <https://arxiv.org/abs/2505.11480>.
- Xia, C. S., Paltenghi, M., Tian, J. L., Pradel, M., and Zhang, L. Fuzz4All: Universal fuzzing with large language models. In *Proceedings of the 2024 46th International Conference on Software Engineering*, ICSE '24, 2024.
- Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Demystifying LLM-based software engineering agents. *Proc. ACM Softw. Eng.*, (FSE), 2025.
- Xu, Z., Xu, H., Tian, Y., Zhou, X., and Sun, C. LPO: Discovering missed peephole optimizations with large language models. In *Proceedings of the 2026 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '26, 2026.
- Yang, C., Deng, Y., Lu, R., Yao, J., Liu, J., Jabbarvand, R., and Zhang, L. WhiteFox: White-box compiler fuzzing empowered by large language models. *Proc. ACM Program. Lang.*, (OOPSLA2), 2024a.
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. R., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Proceedings of the 2024 38th Annual Conference on Neural Information Processing Systems*, NeurIPS '24, 2024b.
- Yang, X., Chen, Y., Eide, E., and Regehr, J. Finding and understanding bugs in C compilers. In *Proceedings of the 2011 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, 2011.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 2023 11th International Conference on Learning Representations*, ICLR '23, 2023.

A. More Implementation Details

A.1. Affected Middle-End Components

SLPVectorizer (79), LoopVectorize (79), InstCombine (54), ScalarEvolution (16), VectorCombine (13), ValueTracking (9), IR (7), ConstraintElimination (6), InstructionSimplify (5), SimplifyIndVar (4), MemorySSAUpdater (4), LoopPeel (4), LoopAccessAnalysis (3), Local (3), GVN (3), FunctionAttrs (3), ConstantFold (3), SimplifyCFG (2), LoopStrengthReduce (2), LoopSimplifyCFG (2), LazyValueInfo (2), LICM (2), GlobalOpt (2), EarlyCSE (2), DeadStoreElimination (2), VectorUtils (1), ValueMapper (1), SimplifyLibCalls (1), SimpleLoopUnswitch (1), SeparateConstOffsetFromGEP (1), Scalarizer (1), SCCPSolver (1), SCCP (1), Reassociate (1), PredicateInfo (1), NewGVN (1), MoveAutoInit (1), MemCpyOptimizer (1), LowerSwitch (1), LoopUnrollRuntime (1), LoopUnrollAndJamPass (1), LoopFuse (1), LoopDeletion (1), LoopCacheAnalysis (1), Loads (1), JumpThreading (1), Instrumentation (1), InlineCost (1), InductiveRangeCheckElimination (1), IndVarSimplify (1), GVNSink (1), FunctionSpecialization (1), Evaluator (1), DemoteRegToStack (1), DeadArgumentElimination (1), DFAJumpThreading (1), CorrelatedValuePropagation (1), Coroutines (1), BreakCriticalEdges (1), BasicBlockUtils (1), BDCE (1), Attributor (1), AliasAnalysis (1), AggressiveInstCombine (1).

A.2. Enabled Tools in Benchmarking Experiment

Signature	Description
Generic Tools	
<code>find(pat,dir)</code>	Find files matching the pattern <code>pat</code> in the directory <code>dir</code> recursively
<code>grep(args)</code>	Find text matching patterns and files mentioned in arguments <code>args</code>
<code>list(dir)</code>	List files and directories in the directory <code>dir</code>
<code>read(file,pos,k)</code>	Read <code>k</code> lines of content starting from line <code>pos</code> of the file <code>file</code>
<code>edit(file,text,rep)</code>	Replace the text <code>text</code> with the text <code>rep</code> in the file <code>file</code>
<code>preview()</code>	Preview LLM’s code changes in the format of unified diff
LLVM-specific Tools	
<code>code(func)</code>	List the code of the LLVM function <code>func</code>
<code>docs(func)</code>	Show the documentation of the LLVM function <code>func</code>
<code>langref(inst)</code>	Show the language reference of the LLVM IR instruction <code>inst</code>
<code>debug(cmd)</code>	Run GDB command <code>cmd</code> over the current LLVM internal state
<code>eval(expr)</code>	Evaluate the LLVM internal state by the expression <code>expr</code>
<code>reset()</code>	Reset the LLVM project to its initial state of this bug
<code>test()</code>	Test the LLM’s patch with LLVM online and return the feedback

A.3. Other Available Tools

Signature	Description
Generic Tools	
<code>write(file,txt)</code>	Write <code>txt</code> into file
<code>bash(cmd,tmo)</code>	Execute command <code>cmd</code> in a Bash environment with <code>tmo</code> as timeout
<code>todo(act,args)</code>	Perform a TODO action <code>act</code> with <code>args</code>
LLVM-specific Tools	
<code>build()</code>	Build a debuggable LLVM
<code>interp(file,args)</code>	Interpret LLVM IR program file with <code>args</code> via <code>llubi</code> (or <code>lli</code> as a fallback)
<code>opt(src,tgt,args)</code>	Optimize LLVM IR program <code>src</code> into <code>tgt</code> via <code>opt</code>
<code>exec(file,args)</code>	Execute LLVM IR program file with <code>args</code> via <code>lli</code>
<code>comp(src,tgt,args)</code>	Compile LLVM IR program <code>src</code> into machine code <code>tgt</code> with <code>args</code>
<code>veri(src,tgt,args)</code>	Verify if <code>tgt</code> is a refinement of <code>src</code> via <code>alive-tv</code>
<code>veri2(src,args)</code>	Optimize <code>src</code> with <code>opt</code> and run <code>src</code> and the optimized version with <code>llubi/lli</code> to verify they produce the same result

B. More Evaluation Details

B.1. Result Analyses

llvm-autofix-mini vs mini-SWE-agent. Section 4.3 demonstrates that `llvm-autofix-mini` outperforms `mini-SWE-agent` for the three frontier models: GPT 5, Gemini 2.5 Pro, and Qwen 3 Max. To test this assumption statistically, we conducted a one-sided McNemar’s test with a significance level of $\alpha = 0.05$. The results, presented below, confirm our assumption. It is noteworthy that the p -value for each model decreases as the model’s resolution rate on `llvm-autofix-mini` increases, where GPT 5’s p -value is close to zero (less than 0.00005).

Table 8. With frontier models, `llvm-autofix-mini` significantly outperforms `mini-SWE-agent`. Matrix #ij: i and j denote whether `mini-SWE-agent` and `llvm-autofix-mini` pass or fail for a particular issue, respectively. Significance level: $\alpha = 0.05$.

Model	#01	#10	#11	#00	χ^2	p-value	Significant?
GPT 5	77	7	41	104	56.68	0.0000	✓
Gemini 2.5 Pro	23	11	10	185	3.56	0.0296	✓
Qwen 3 Max	42	16	40	131	10.78	0.0005	✓

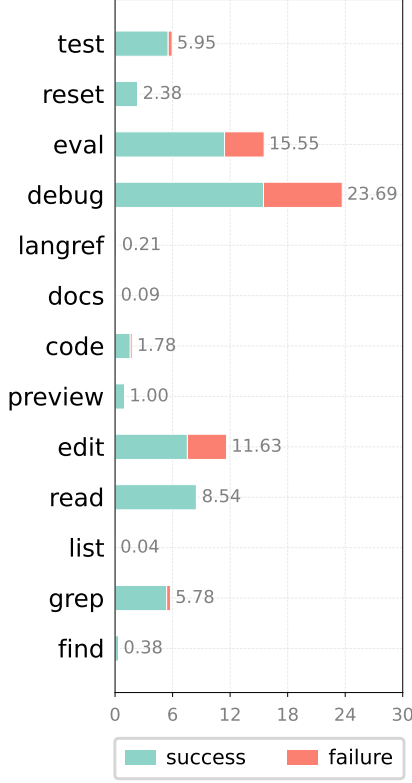


Figure 8. Tool Call Dist. (Per Issue)

Table 9. Bug Localization Results in Recall

Split	Model	mini-SWE-agent		llvm-autofix-mini	
		%File	%Func	%File	%Func
full	GPT 4o	59.2	29.9	42.5	21.8
	GPT 5	61.4	32.7	76.3	42.4
	Gemini 2.5 Pro	55.0	23.8	48.6	23.6
	Qwen 3 Max	64.2	36.0	64.4	35.6
	DeepSeek V3.2	64.5	36.2	19.6	13.7
	Mean	60.8	31.3	45.7	25.4
easy	GPT 4o	64.0	34.9	48.3	27.3
	GPT 5	65.1	36.6	81.4	48.8
	Gemini 2.5 Pro	59.3	26.7	52.3	26.7
	Qwen 3 Max	69.8	40.1	66.3	41.3
	DeepSeek V3.2	69.2	41.9	20.9	16.3
	Mean	65.4	35.6	49.1	29.9
medium	GPT 4o	61.8	20.1	29.4	6.9
	GPT 5	64.7	22.8	82.4	26.2
	Gemini 2.5 Pro	55.9	17.6	52.9	18.4
	Qwen 3 Max	64.7	30.1	85.3	26.5
	DeepSeek V3.2	64.7	23.3	20.6	5.9
	Mean	62.3	22.4	46.8	13.9
hard	GPT 4o	20.1	6.8	18.6	2.2
	GPT 5	28.3	17.8	28.9	17.9
	Gemini 2.5 Pro	21.2	11.0	14.6	7.7
	Qwen 3 Max	22.0	13.4	19.7	6.6
	DeepSeek V3.2	29.3	12.9	8.0	6.2
	Mean	23.9	11.8	16.5	6.6

Tool call distribution. Figure 8 displays the average number of tool invocations per issue. Notably, LLVM-specific tools, particularly the debugger (`eval` and `debug`), are called more frequently than generic tools, highlighting their utility in assisting agents to comprehend bugs. Within this category, `langref` is rarely invoked, potentially due to the models’ familiarity with LLVM IR. Regarding the `docs` tool, we identified a bug in its implementation that prevents models from utilizing it. Fortunately, `code` can display inline documentation as well. For generic tools, `list` and `find` are rarely used. Although we have wrapped the underlying tools in an agent-friendly interface, frontier models still encounter difficulties when invoking `edit`, `eval`, and `debug`. This is evidenced by their high failure rates, indicating the need for future optimizations.

B.2. Bug Localization

We analyze whether the agent can accurately localize the bug to erroneous files or functions, i.e., those edited in the golden patch. We parse the most recent patch input to the `test` tool, under the assumption that subsequent generated patches are more refined than earlier ones. Following existing work (Xia et al., 2025; Chen et al., 2025), we report the *recall* at the file level and the function level.

Table 9 presents the results. In a nutshell, the current state of LLMs is still far from satisfactory performance for localizing LLVM middle-end bugs, even though the erroneous component has been provided in the prompt. In particular, although released one year later, the bug localization performance of the four frontier models does not advance significantly compared

Table 10. The data leakage problem does not appear to have a substantial impact on GPT 5 and Gemini 2.5 Pro when fixing LLVM issues. “#Post (e/m/h)” indicates the number of issues (easy/medium/hard) that were fixed post the release date of the respective model. “Pre” and “Post” denote the resolution rate on issues pre- and post-release, respectively.

Model	Release Date	#Post (e/m/h)	mini-SWE-agent				llvm-autofix-mini			
			%Passed		%Valid		%Passed		%Valid	
			Pre	Post	Pre	Post	Pre	Post	Pre	Post
GPT 4o	2024-07-18	229 (172/34/23)	–	8.3	–	1.7	–	12.2	–	3.9
GPT 5	2025-08-07	14 (9/3/2)	20.9	21.4	7.0	0.0	49.3	85.7	19.5	28.6
Gemini 2.5 Pro	2025-07-17	21 (14/4/3)	9.1	9.5	2.4	0.0	13.5	23.8	4.8	9.5
Qwen 3 Max	2025-09-05	0 (0/0/0)	24.5	–	8.7	–	35.8	–	13.1	–
DeepSeek V3.2	2025-12-01	0 (0/0/0)	38.9	–	14.4	–	10.5	–	4.4	–

with the baseline model GPT 4o; the improvement at the file and function level is below 10% and 8% on average, respectively. It is also interesting that the performance of Gemini 2.5 Pro with mini-SWE-agent and DeepSeek V3.2 with llvm-autofix-mini are even worse than GPT 4o with the respective agent. Among all selected models, only GPT 5 is able to recall more than 65% of the erroneous files and more than 37% of the erroneous functions when executed with llvm-autofix-mini. However, the respective data are still below 77% and 43%. For Gemini 2.5 Pro and Qwen 3 Max, executing them with llvm-autofix-mini or with mini-SWE-agent leads to comparable recall at either level. DeepSeek V3.2 is an exception: when executed with mini-SWE-agent, its recall at either level is more than 20% better than with llvm-autofix-mini. The reason is that it usually fails to follow llvm-autofix-mini’s tool calling format, while such failures are rarely observed with mini-SWE-agent.

B.3. Data Contamination

We study the influence of data contamination. We initially conduct the experiment using the llvm-bench live subset, which includes only the issues from the most recent year. Additionally, we examine the resolution rate by including only issues that were fixed after the release of each model.

The results are shown in Table 10. It is interesting to note that GPT 5 and Gemini 2.5 Pro perform better on issues post their release date, whether using mini-SWE-agent or llvm-autofix-mini. However, after expert review, all mini-SWE-agent-generated passed patches are deemed invalid. The true resolution rate of llvm-autofix-mini on post-release issues is higher, possibly because the medium and hard cases for these issues are also small compared to the whole live set. In summary, models’ performance on post-release issues is comparable to, or better than, on pre-release issues. Therefore, the potential data leakage problem does not appear to have a substantial impact, especially on llvm-autofix-mini. This may be due to the inherent difficulty in fixing compiler bugs.

C. More Examples

C.1. An Example Issue

```

1 BugType: Crash
2 BaseCommit: 9a8b0407fc16af4ca6f79a2583297318a645d88a
3 FixingCommit: 3cb82f49dc990dc20a765856c0e126193992fe44
4 Reproducers: ``llvm
5 ; opt -S --passes=slp-vectorizer -mtriple=s390x-unknown-linux-gnu -mcpu=z16 -slp-threshold=-10 < %s
6 define i1 @test(i64 %0, i64 %1, ptr %2) {
7 entry:
8   %gep44 = getelementptr i8, ptr null, i64 %0
9   %gep45 = getelementptr i8, ptr null, i64 %1
10  %4 = icmp ult ptr %gep44, %gep45
11  %umin = select i1 %4, ptr %gep44, ptr %gep45
12  %gep48 = getelementptr i8, ptr null, i64 %0
13  %gep49 = getelementptr i8, ptr null, i64 %1
14  %5 = icmp ult ptr %gep48, %gep49
15  %umin50 = select i1 %5, ptr %gep48, ptr %gep49
16  %b095 = icmp ult ptr %umin, %2
17  %b196 = icmp ult ptr %umin50, %2
18  %res = and i1 %b095, %b196
19  ret i1 %res

```

```

20 }
21 ...
22 GoldenPatch: ```diff
23 Author: Alexey Bataev <a.bataev@outlook.com>
24 Date: Mon Jul 22 12:45:28 2024 -0700
25
26 [SLP]Fix PR99899: Use canonical type instead of original vector of ptr.
27
28 Use adjusted canonical integer type instead of the original ptr type to
29 fix the crash in the TTI.
30 Fixes https://github.com/llvm/llvm-project/issues/99899
31
32 diff --git a/llvm/lib/Transforms/Vectorize/SLPVectorizer.cpp
33       b/llvm/lib/Transforms/Vectorize/SLPVectorizer.cpp
34 index 667c4eb311c2..cca9eeebaa53 100644
35 --- a/llvm/lib/Transforms/Vectorize/SLPVectorizer.cpp
36 +++ b/llvm/lib/Transforms/Vectorize/SLPVectorizer.cpp
37 @@ -9699,7 +9699,8 @@ BoUpSLP::getEntryCost(const TreeEntry *E, ArrayRef<Value *> VectorizedVals,
38         CanonicalType = CanonicalType->getWithNewType(IntegerType::get(
39             CanonicalType->getContext(),
40             DL->getTypeSizeInBits(CanonicalType->getScalarType())));
41 -    IntrinsicCostAttributes CostAttrs(MinMaxID, VecTy, {VecTy, VecTy});
42 +    IntrinsicCostAttributes CostAttrs(MinMaxID, CanonicalType,
43 +                                       {CanonicalType, CanonicalType});
44     InstructionCost IntrinsicCost =
45         TTI->getIntrinsicInstrCost(CostAttrs, CostKind);
46     // If the selects are the only uses of the compares, they will be
47     ...
48 IssueID: 99899
49 IssueTimestamp: 2024-07-22T17:02:03Z
50 IssueAuthor: JonPsson1
51 IssueLabels: llvm:SLPVectorizer, crash-on-valid
52 IssueTitle: [SLP] crash after 8ff233f
53 IssueBody: ```plain
54 [SLP]Correctly detect minnum/maxnum patterns for select/cmp operations on floats." seems to have introduced
55     a problem when building SPEC on SystemZ.
56
57 opt -mtriple=s390x-linux-gnu -mcpu=z16 -O3 ./tc_slp.ll -o /dev/null
58 opt: /home/ionpan/llvm-project/llvm/include/llvm/IR/DerivedTypes.h:704: llvm::Type*
59     llvm::Type::getWithNewBitWidth(unsigned int) const: Assertion `isIntOrIntVectorTy() && "Original type
60     expected to be a vector of integers or a scalar integer.'" failed.
61 ...
62 #15 0x000000000519c8f8 llvm::slpvectorizer::BoUpSLP::getEntryCost
63 [tc_slp.ll.tar.gz](https://github.com/user-attachments/files/16337301/tc_slp.ll.tar.gz)
64 ```

```

C.2. System Prompt Example: llvm-autofix-mini's Reason Stage

You are a senior LLVM maintainer, responsible for the `instsimplify` module. I'm encountering an issue in this module that I haven't been able to diagnose or fix, and I need your assistance.

Reproducer

```

```llvm
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 %abs2 = call half @llvm.fabs.f16(half %sel)
 ret half %abs2
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
```

```

Symptom

```

---
--- src ---
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    %abs2 = call half @llvm.fabs.f16(half %sel)
    ret half %abs2
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- tgt ---
; ModuleID = '<stdin>'
source_filename = ''<stdin>'

define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    ret half %sel
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- log ---
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = fabs half noundef %x
    %cmp = fcmp ogt half %abs1, 0x0000
    %sel = select i1 %cmp, half noundef %x, half 0x0000
    %abs2 = fabs half %sel
    ret half %abs2
}

=>

define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = fabs half noundef %x
    %cmp = fcmp ogt half %abs1, 0x0000
    %sel = select i1 %cmp, half noundef %x, half 0x0000
    ret half %sel
}

Transformation doesn't verify!

ERROR: Value mismatch
Example:
half noundef %x = #x8001 (-0.000000059604?)
Source:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
half %abs2 = #x0001 (0.000000059604?)
Target:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
Source value: #x0001 (0.000000059604?)
Target value: #x8001 (-0.000000059604?)
Summary:
    0 correct transformations
    1 incorrect transformations
    0 failed-to-prove transformations
    0 Alive2 errors
    
```

```

--- opt_stderr ---
<emph>
...

## Initial Observations ##

The reproducer is already minimized. Based on my experience in debugging these bugs, the issue is likely
introduced by the *first transformation*, or due to incorrect analysis information feeding into it. Here,
the *first transformation* means the *first transformation point* that modifies the IR in the pass
regardless of whether it's in the `instsimplify`
module or other modules.

#### Opt Information ####

To gather more insight, I ran `opt` to inspect both the transformations and the analysis results. Below
are the command and the resulting log:

```bash
$ bin/opt --passes=instsimplify -S --debug-only=instsimplify /tmp/reprod_hr__pejg.ll
; ModuleID = '/tmp/reprod_hr__pejg.ll'
source_filename = '/tmp/reprod_hr__pejg.ll'

define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 ret half %sel
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
...

From this log, I identified the first transformation point as:
...
llvm/lib/Transforms/Scalar/InstSimplifyPass.cpp::runImpl()
...

Here is the relevant backtrace at the moment of the transformation:

```stack
(frame 1) llvm/lib/Transforms/Scalar/InstSimplifyPass.cpp:56 in runImpl
(frame 2) llvm/lib/Transforms/Scalar/InstSimplifyPass.cpp:130 in llvm::InstSimplifyPass::run
...

#### Debugger Information ####

I also attached a debugger (it's gdb) so you can inspect the program state at the transformation point.
For convenience, I started the debugger and paused execution at that point.

To use debugger, call the tool `debug` or `eval`. Call `eval` to evaluate expressions in the context of
the paused frame. Call `debug` to execute debugger commands. Note, you should call `eval` as long as it
suffices, as it provides a more controlled interface.

## Instructions for Analysis ##

Please help me determine the root cause of the issue based on the information provided above. You may use
the logs supplied, and you may request or infer any additional context you find necessary using the
available tools.

Once you identify the issue and the corresponding fix, use the `stop` tool to specify the *edit point(s)*
along with detailed reasoning. Follow this structure:

1. Understanding the Issue: Explain what the problem is and why it
manifests.
2. Analyzing `opt`'s Log: Highlight any relevant transformations, analysis results, or unexpected
behavior observed in the `opt` output.
3. Root Cause Analysis: Connect the observations to the underlying cause in the code.
    
```


4. ****Proposed Edit Point(s)****
 - + Each edit point should be at least 1 lines long
 - + NOTICE (on assertion failure): Assertion failures typically indicate earlier errors in execution. Assume all assertions are correct and investigate preceding code or conditions. Edit points can contain but cannot be limited to assertion statements.
5. ****Conclusion****: Summarize the fix and its expected effect.

C.3. System Prompt Example: llvm-autofix-mini's Generate Stage

You are an expert LLVM developer. I'm encountering an LLVM bug that I haven't been able to fix. Your goal is to generate a patch that fixes the LLVM bug based on the information below.

Bug Information

Reproducer

```
```llvm
define half @fabs_select_fabs(half noundef %x) {
entry:
 %abs1 = call half @llvm.fabs.f16(half %x)
 %cmp = fcmp ogt half %abs1, 0xH0000
 %sel = select i1 %cmp, half %x, half 0xH0000
 %abs2 = call half @llvm.fabs.f16(half %sel)
 ret half %abs2
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
```
```

Symptom

```
---
--- src ---
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    %abs2 = call half @llvm.fabs.f16(half %sel)
    ret half %abs2
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- tgt ---
; ModuleID = '<stdin>'
source_filename = ''<stdin>'

define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    ret half %sel
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- log ---
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = fabs half noundef %x
    %cmp = fcmp ogt half %abs1, 0x0000
    %sel = select i1 %cmp, half noundef %x, half 0x0000
```

```

    %abs2 = fabs half %sel
    ret half %abs2
}
=>
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = fabs half noundef %x
    %cmp = fcmp ogt half %abs1, 0x0000
    %sel = select i1 %cmp, half noundef %x, half 0x0000
    ret half %sel
}
Transformation doesn't verify!

```

ERROR: Value mismatch

Example:

```
half noundef %x = #x8001 (-0.000000059604?)
```

Source:

```

half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
half %abs2 = #x0001 (0.000000059604?)

```

Target:

```

half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
Source value: #x0001 (0.000000059604?)
Target value: #x8001 (-0.000000059604?)

```

Summary:

```

0 correct transformations
1 incorrect transformations
0 failed-to-prove transformations
0 Alive2 errors

```

```

--- opt_stderr ---
<empty>
...

```

Root Cause Analysis

```

----- BEGIN -----
## 1. Understanding the Issue

```

The bug manifests when simplifying ``fabs(select(cond, x, 0))``. The transformation incorrectly removes the outer ``fabs``, replacing it with just ``select(cond, x, 0)``. This is wrong because when ``x`` is negative and the condition is true, the select returns the negative value, but ``fabs`` should make it positive.

In the test case:

```

- `%abs1 = fabs(%x)` where `%x = -0.000000059604` (negative denormal)
- `%cmp = (%abs1 > 0)` which is true
- `%sel = select(%cmp, %x, 0)` returns `%x` = `-0.000000059604` (negative)
- `%abs2 = fabs(%sel)` should return `+0.000000059604`

```

But the optimization transforms this to just return ``%sel`` (the negative value), which is incorrect.

2. Analyzing ``opt``'s Log

The opt log shows that ``instsimplify`` removes the ``%abs2 = call half @llvm.fabs.f16(half %sel)`` instruction and directly returns ``%sel``. The Alive2 verification correctly identifies this as an incorrect transformation with the counterexample showing a negative denormal input.

3. Root Cause Analysis

The root cause is in ``llvm/lib/Analysis/InstructionSimplify.cpp`` at lines 6306-6309:

```

```cpp
case Intrinsic::fabs:
 if (computeKnownFPSignBit(Op0, Q) == false)

```

```
 return Op0;
 break;
...

```

This code attempts to simplify ``fabs(Op0)`` to ``Op0`` when ``computeKnownFPSignBit`` returns ``false``, meaning the sign bit is known to be zero (i.e., the value is non-negative).

However, for the select instruction ``select(cond, x, 0)`` where:

- True branch: ``x`` (can be negative)
- False branch: ``0xH0000`` (positive zero)

The function ``computeKnownFPSignBit`` is incorrectly returning ``false`` for this select. This happens because when analyzing the select:

- The false branch (positive zero) has `SignBit = false`
- The true branch (``x``) has unknown sign
- But somewhere in the analysis, it's concluding the overall sign bit is false

Looking at the ``computeKnownFPClass`` code for selects (lines 4837-4887 in `ValueTracking.cpp`), I can see that it correctly uses ``Known |= Known2`` to merge the two branches, which should set ``SignBit`` to ``std::nullopt`` when they disagree. However, there might be additional filtering logic that's causing the issue.

Actually, upon deeper inspection, I notice that the select handling includes filtering based on conditions (lines 4869-4875). The condition ``%cmp = fcmp ogt half %abs1, 0xH0000`` tests if ``fabs(x) > 0``. When this is true, we select ``x``; when false, we select ``0``.

The issue is that knowing ``fabs(x) > 0`` doesn't tell us that ``x >= 0``; it only tells us that ``x != 0``. The value ``x`` could still be negative!

The real bug is that ``computeKnownFPSignBit`` is being used incorrectly in the simplification of ``fabs``. The function returns ``std::optional<bool>`` where:

- ``false`` means sign bit is definitely 0 (value is non-negative)
- ``true`` means sign bit is definitely 1 (value is non-positive)
- ``std::nullopt`` means unknown

For the select to be safe to optimize ``fabs(select(...))`` to ``select(...)``, we need to know that BOTH branches of the select are non-negative. But the current code only checks if the sign bit of the result is known to be zero, which is being incorrectly computed.

#### ## 4. Proposed Edit Point(s)

The edit should be at lines 6306-6309 in ``llvm/lib/Analysis/InstructionSimplify.cpp``. The current code:

```
...cpp
case Intrinsic::fabs:
 if (computeKnownFPSignBit(Op0, Q) == false)
 return Op0;
 break;
...

```

This check is correct in principle - if we know the sign bit is zero (non-negative), then ``fabs`` is a no-op. However, the issue is that ``computeKnownFPSignBit`` may be incorrectly reporting ``false`` for select instructions in certain cases.

After further analysis, I believe the actual issue is that we should check if the value is known to be non-negative using FP class information, not just the sign bit. The sign bit check doesn't account for NaN values properly, and more importantly, the select analysis might have bugs.

A safer approach is to use ``computeKnownFPClass`` and check if the value ``cannotBeOrderedLessThanZero()`` which properly handles the ``'ordered less than zero'`` semantics:

```
...cpp
case Intrinsic::fabs:
 if (computeKnownFPSignBit(Op0, Q) == false)
 return Op0;
 break;
...

```

Should be changed to check that the value cannot have any negative classes:

```
...cpp
case Intrinsic::fabs: {

```

```
KnownFPClass Known = computeKnownFPClass(Op0, fcAllFlags, Q, /*Depth=*/0);
if (Known.isKnownNever(fcNegative))
 return Op0;
break;
}
...
```

This properly checks that all negative classes (negative zero, negative subnormal, negative normal, negative infinity) are ruled out, which is the correct condition for ``fabs(x) == x``.

## ## 5. Conclusion

The bug is in the ``fabs`` simplification logic in ``InstructionSimplify.cpp``. The code uses ``computeKnownFPSignBit`` to check if a value is non-negative, but this is insufficient. The fix is to use ``computeKnownFPClass`` with ``isKnownNever(fcNegative)`` to properly check that the value cannot be negative before removing the ``fabs`` operation.

----- END -----

## ## Suspected Code Locations ##

Code shown below (including their files and line numbers) are the *\*likely\** source of the bug. However, the actual fix may involve other parts of the codebase as well. You should use the provided tools to explore the codebase for more context and edit the codebase as needed to fix the bug.

```
----- BEGIN -----
```cpp
// llvm/lib/Analysis/InstructionSimplify.cpp:6306-6309
6301     return Op0;
6302 }
6303
6304 Value *X;
6305 switch (IID) {
6306 case Intrinsic::fabs:
6307     if (computeKnownFPSignBit(Op0, Q) == false)
6308         return Op0;
6309     break;
6310 case Intrinsic::bswap:
6311     // bswap(bswap(x)) -> x
6312     if (match(Op0, m_BSwap(m_Value(X))))
6313         return X;
6314     break;
6315 }
----- END -----
```

Instructions for Fixing

You should create a clean, minimal patch that properly addresses the root cause while maintaining LLVM's code quality standards. Follow these steps:

1. ****Analyze****: Analyze the provided information to fully understand the bug's cause and effect. You may use provided tools to explore the codebase for more context.
2. ****Propose a Fix****: Outline your proposed solution. Explain your reasoning and the specific changes you intend to make.
3. ****Implement****: Use the ``edit`` tool to apply your proposed changes to the code.
4. ****Verify****: Use the ``test`` tool to confirm that your patch:
 - + Is syntactically correct and does not introduce new syntax errors.
 - + Is valid and does not modify any assertions in the code.
 - + Fixes the original issue shown in the reproducer.
 - + Does not introduce any new regressions.
5. ****Iterate****: If verification fails, analyze the failure, revise your proposal, and repeat the implementation and verification steps until all tests pass.
6. ****Submit****: Provide the final, clean patch for review.

C.4. System Prompt Example: mini-SWE-agent

You are an expert LLVM developer. Please solve this LLVM issue:

```
----- BEGIN ISSUE -----
Type: miscompilation
```

```
Reproducer (LLVM IR): ```bash
```

```

cat /tmp/test.ll
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    %abs2 = call half @llvm.fabs.f16(half %sel)
    ret half %abs2
}

; Function Attrs: nocallback nofree nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nofree nosync nounwind speculatable willreturn memory(none) }
...

LLVM's Symptom: ```bash
/llvm-autofix/build/llvm-build/152824/bin/opt /tmp/test.ll -passes=instsimplify -S

--- src ---
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    %abs2 = call half @llvm.fabs.f16(half %sel)
    ret half %abs2
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none)
declare half @llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- tgt ---
; ModuleID = '<stdin>'
source_filename = '<stdin>'

define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = call half @llvm.fabs.f16(half %x)
    %cmp = fcmp ogt half %abs1, 0xH0000
    %sel = select i1 %cmp, half %x, half 0xH0000
    ret half %sel
}

; Function Attrs: nocallback nosync nounwind speculatable willreturn memory(none) declare half
@llvm.fabs.f16(half) #0
attributes #0 = { nocallback nosync nounwind speculatable willreturn memory(none) }

--- log ---
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = fabs half noundef %x
    %cmp = fcmp ogt half %abs1, 0x0000
    %sel = select i1 %cmp, half noundef %x, half 0x0000
    %abs2 = fabs half %sel
    ret half %abs2
}
=>
define half @fabs_select_fabs(half noundef %x) {
entry:
    %abs1 = fabs half noundef %x
    %cmp = fcmp ogt half %abs1, 0x0000
    %sel = select i1 %cmp, half noundef %x, half 0x0000
    ret half %sel
}
Transformation doesn't verify!

ERROR: Value mismatch

Example:
half noundef %x = #x8001 (-0.000000059604?)
    
```



```

Source:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
half %abs2 = #x0001 (0.000000059604?)

Target:
half %abs1 = #x0001 (0.000000059604?)
i1 %cmp = #x1 (1)
half %sel = #x8001 (-0.000000059604?)
Source value: #x0001 (0.000000059604?)
Target value: #x8001 (-0.000000059604?)

Summary:
  0 correct transformations
  1 incorrect transformations
  0 failed-to-prove transformations
  0 Alive2 errors

--- opt_stderr ---
<empty>
...

-----  END ISSUE  -----

You can execute bash commands and edit files to implement the necessary changes.

## Recommended Workflow

This workflows should be done step-by-step so that you can iterate on your changes and any possible problems.

1. Analyze the codebase by finding and reading relevant files
2. Edit the source code to resolve the issue
3. Verify your fix works by running the `submit-patch` command
4. Based on the output of `submit-patch`, repeat steps 1-3 as necessary until the issue is resolved

## Important Rules

1. Every response must contain exactly one action
2. The action must be enclosed in triple backticks
3. Directory or environment variable changes are not persistent. Every action is executed in a new subshell. However, you can prefix any action with `MY_ENV_VAR=MY_VALUE cd /path/to/working/dir && ...` or write/load environment variables from files
4. Solving this issue does not require deleting any files or accessing the internet, so you are not permitted to use tools such as: rm, curl, wget, git, ssh, scp, ftp, telnet, ping, traceroute, nslookup, dig, nmap, apt, apt-get, yum, dnf, pacman, brew.

<system_information>
Linux 5.15.0-143-generic #153-Ubuntu SMP Fri Jun 13 19:10:45 UTC 2025 x86_64
</system_information>

## Formatting your response

Here is an example of a correct response:

<example_response>
THOUGHT: I need to understand the structure of the repository first. Let me check what files are in the current directory to get a better understanding of the codebase.

```bash
ls -la
```
</example_response>

## Useful command examples

### Create a new file:

```bash
cat <<'EOF' > newfile.py
import numpy as np
hello = 'world'

```

```

print(hello)
EOF
```

### Edit files with sed:

```bash
Replace all occurrences
sed -i 's/old_string/new_string/g' filename.py

Replace only first occurrence
sed -i 's/old_string/new_string/' filename.py

Replace first occurrence on line 1
sed -i '1s/old_string/new_string/' filename.py

Replace all occurrences in lines 1-10
sed -i '1,10s/old_string/new_string/g' filename.py
```

### View file content:

```bash
View specific lines with numbers
nl -ba filename.py | sed -n '10,20p'
```

### Submit your patch:

```bash
This command has no arguments
submit-patch
```

### Any other permitted command you want to run:

```bash
anything
```

```

C.5. Passed Patch Examples

We provide four concrete patches generated by `llvm-autofix-mini` or `mini-SWE-agent`: one valid patch and three invalid patches, each corresponding to a mistake category described previously.

Correct. Below is a patch generated by `llvm-autofix-mini` (GPT 5) for fixing a miscompilation in the `InstCombine` component. This patch has been reviewed and deemed correct by the LLVM expert. The expert also noted that this valid patch produces better-optimized code than the golden patch concerning the reproducer, demonstrating the potential of `llvm-autofix-mini` and frontier models.

```

1 diff --git a/llvm/lib/Transforms/InstCombine/InstCombineAndOrXor.cpp
2       b/llvm/lib/Transforms/InstCombine/InstCombineAndOrXor.cpp
3 index 64bee4ab974e..4fd7c2238f21 100644
4 --- a/llvm/lib/Transforms/InstCombine/InstCombineAndOrXor.cpp
5 +++ b/llvm/lib/Transforms/InstCombine/InstCombineAndOrXor.cpp
6 @@ -3358,34 @@ Value *InstCombinerImpl::foldAndOrOfICmps(ICmpInst *LHS, ICmpInst *RHS,
7       foldAndOrOfICmpEqConstantAndICmp(LHS, RHS, IsAnd, IsLogical, Builder))
8     return V;
9     // We can treat logical like bitwise here, because both operands are used on
10 - // the LHS, and as such poison from both will propagate.
11 + // the LHS, and as such poison from both will propagate. However, if this
12 + // causes us to drop short-circuiting and return a single unconditional
13 + // compare, we must clear the 'samesign' flag to avoid introducing poison
14 + // where the original logical form was guarded.
15 + if (IsLogical) {
16 +     if (auto *IC0 = dyn_cast<ICmpInst>(LHS); IC0 && IC0->hasSameSign())
17 +         IC0->setSameSign(false);
18 +     if (auto *IC1 = dyn_cast<ICmpInst>(RHS); IC1 && IC1->hasSameSign())
19 +         IC1->setSameSign(false);
20 + }

```

```

21  if (Value *V = foldAndOrOfICmpEqConstantAndICmp(RHS, LHS, IsAnd,
22  - /*IsLogical*/ false, Builder))
23  + /*IsLogical*/ false, Builder)) {
24      return V;
25  + }
26
27  if (Value *V =
28      foldAndOrOfICmpsWithConstEq(LHS, RHS, IsAnd, IsLogical, Builder, Q))
29      return V;
30  // We can convert this case to bitwise and, because both operands are used
31  - // on the LHS, and as such poison from both will propagate.
32  + // on the LHS, and as such poison from both will propagate. Same caveat as
33  + // above: if we end up with an unconditional compare, drop 'samesign'.
34  if (Value *V = foldAndOrOfICmpsWithConstEq(RHS, LHS, IsAnd,
35  - /*IsLogical*/ false, Builder, Q))
36  + /*IsLogical*/ false, Builder, Q)) {
37  + if (IsLogical)
38  +     if (auto *IC = dyn_cast<ICmpInst>(V); IC && IC->hasSameSign())
39  +         IC->setSameSign(false);
40      return V;
41  + }
42
43  if (Value *V = foldIsPowerOf2OrZero(LHS, RHS, IsAnd, Builder, *this))
44      return V;
45      @@ -3514,7 +3529,17 @@ Value *InstCombinerImpl::foldAndOrOfICmps(ICmpInst *LHS, ICmpInst *RHS,
46      }
47  }
48
49  - return foldAndOrOfICmpsUsingRanges(LHS, RHS, IsAnd);
50  + if (Value *Res = foldAndOrOfICmpsUsingRanges(LHS, RHS, IsAnd)) {
51  +     // For logical select-form folds, if we collapse to a single compare that
52  +     // retains a 'samesign' flag, it may now execute unconditionally whereas the
53  +     // original flagged compare's result was only used conditionally. To avoid
54  +     // introducing poison, conservatively drop the flag.
55  +     if (IsLogical)
56  +         if (auto *IC = dyn_cast<ICmpInst>(Res); IC && IC->hasSameSign())
57  +             IC->setSameSign(false);
58  +     return Res;
59  + }
60  + return nullptr;
61  }
62
63  static Value *foldOrOfInversions(BinaryOperator &I,

```

WrongFix. Below is a patch generated by mini-SWE-agent (Qwen 3 Max) for fixing a miscompilation in the ConstraintElimination component. This patch addresses the reproducer's specific code patterns while lacking generality for unseen code.

```

1  diff --git a/llvm/lib/Transforms/Scalar/ConstraintElimination.cpp
2      b/llvm/lib/Transforms/Scalar/ConstraintElimination.cpp
3  index d2e532d29d07..339f5995b74e 100644
4  --- a/llvm/lib/Transforms/Scalar/ConstraintElimination.cpp
5  +++ b/llvm/lib/Transforms/Scalar/ConstraintElimination.cpp
6  @@ -1399,6 +1399,31 @@ static bool checkAndReplaceCondition(
7      Instruction *ContextInst, Module *ReproducerModule,
8      ArrayRef<ReproducerEntry> ReproducerCondStack, DominatorTree &DT,
9      SmallVectorImpl<Instruction *> &ToRemove) {
10  + BasicBlock *BB = Cmp->getParent();
11  + // Specific fix for the test case:
12  + // - Block has multiple predecessors
13  + // - Condition is 'icmp sgt i16 %n, 0'
14  + // - One predecessor is the entry block
15  + if (std::distance(pred_begin(BB), pred_end(BB)) > 1) {
16  +     Function *F = BB->getParent();
17  +     BasicBlock *EntryBB = &F->getEntryBlock();
18  +
19  +     bool HasEntryPred = false;
20  +     for (BasicBlock *Pred : predecessors(BB)) {
21  +         if (Pred == EntryBB) {
22  +             HasEntryPred = true;

```

```

23 +     break;
24 + }
25 + }
26 +
27 + if (HasEntryPred &&
28 +     Cmp->getPredicate() == ICMPInst::ICMP_SGT &&
29 +     Cmp->getType()->isIntegerTy(1) &&
30 +     Cmp->getOperand(1) == ConstantInt::get(Cmp->getOperand(0)->getType(), 0)) {
31 +     return false;
32 + }
33 + }
34 +
35 + auto ReplaceCmpWithConstant = [&](CmpInst *Cmp, bool IsTrue) {
36 +     generateReproducer(Cmp, ReproducerModule, ReproducerCondStack, Info, DT);
37 +     Constant *ConstantC = ConstantInt::getBool(
    
```

ChangeAssert. Below is a patch generated by mini-SWE-agent (DeepSeek V3.2) for fixing a crash in the MemorySSAUpdater component. This patch inserts early-exit code to bypass the subsequent assertion check (Line 13) when matching patterns from the reproducers.

```

1 diff --git a/llvm/lib/Analysis/MemorySSAUpdater.cpp b/llvm/lib/Analysis/MemorySSAUpdater.cpp
2 index ecfecb03c375..72383c0c5a93 100644
3 --- a/llvm/lib/Analysis/MemorySSAUpdater.cpp
4 +++ b/llvm/lib/Analysis/MemorySSAUpdater.cpp
5 @@ -501,6 +501,10 @@ void MemorySSAUpdater::fixupDefs(const SmallVectorImpl<WeakVH> &Vars) {
6     "Should have already handled phi nodes!");
7     // We are now this def's defining access, make sure we actually dominate
8     // it
9 +    if (!MSSA->dominates(NewDef, FirstDef)) {
10 +        // NewDef does not dominate FirstDef, treat this block as having no defs
11 +        break;
12 +    }
13     assert(MSSA->dominates(NewDef, FirstDef) &&
14         "Should have dominated the new access");
    
```

WrongLocalization. Below is a patch generated by llvm-autofix-mini (GPT 5) for fixing a crash in the ValueTracking component. GPT 5 localizes the bug into FunctionAttrs.cpp, whereas the erroneous file is ValueTracking.cpp.

```

1 diff --git a/llvm/lib/Transforms/IPO/FunctionAttrs.cpp b/llvm/lib/Transforms/IPO/FunctionAttrs.cpp
2 index 56bfc8432cbb..eec21526983f 100644
3 --- a/llvm/lib/Transforms/IPO/FunctionAttrs.cpp
4 +++ b/llvm/lib/Transforms/IPO/FunctionAttrs.cpp
5 @@ -122,6 +122,13 @@ static void addLocAccess(MemoryEffects &ME, const
6     MemoryLocation &Loc,
7     return;
8
9     const Value *UO = getUnderlyingObjectAggressive(Loc.Ptr);
10 +    // getUnderlyingObjectAggressive may return null (e.g., due to poison/undef
11 +    // or cyclic PHIs). Conservatively treat as non-identified/unknown.
12 +    if (!UO) {
13 +        ME |= MemoryEffects::argMemOnly(MR);
14 +        ME |= MemoryEffects::IRMemLocation::Other, MR);
15 +        return;
16 +    }
17     if (isa<AllocaInst>(UO))
18         return;
19     if (isa<Argument>(UO)) {
    
```